

Technical paper

An adaptive genetic algorithm based on Q-learning for energy-efficient e-waste disassembly line balancing and rebalancing considering task failures

Kaipu Wang^a, Xiaoyi Ma^a, Yibing Li^a, Yabo Luo^{a,*}, Yingli Li^b, Liang Gao^c

^a Hubei Digital Manufacturing Key Laboratory, School of Mechanical and Electronic Engineering, Wuhan University of Technology, Wuhan 430070, China

^b School of Management, Wuhan University of Science and Technology, Wuhan 430081, China

^c State Key Laboratory of Intelligent Manufacturing Equipment and Technology, School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China

ARTICLE INFO

Keywords:

E-waste
Disassembly line balancing
Task failure
Rebalancing
Genetic algorithm
Q-learning

ABSTRACT

The efficient disassembly and recycling of e-waste not only provides economic benefits but also contributes to reducing energy consumption. However, the disassembly process is often influenced by uncertainties, such as damage or deformation of components, which may result in potential task failures. These failures can disrupt the balance of the disassembly line, affecting the efficiency of subsequent tasks. Therefore, it is crucial to develop a decision-making model and optimization method to address disassembly failures. This study presents a predictive disassembly line balancing model with objectives focused on the number of workstations, the smoothness index, and energy consumption. The optimization objective of adjusting the disassembly sequence is introduced, and a rebalancing model is developed to reallocate the remaining tasks in response to various failures. The sequence combination that minimizes comprehensive energy consumption is selected as the optimal disassembly strategy. Considering the complexity and dynamic disturbance of the problem, an adaptive multi-objective genetic algorithm based on Q-learning is proposed. To improve the quality of the disassembly solutions, six evolutionary actions and four population performance states are designed. During the algorithm's iteration, the search strategy is dynamically adjusted through Q-learning. The effectiveness of the proposed algorithm is verified by solving several classic disassembly cases and comparing the results with those from six advanced algorithms. Finally, in an actual refrigerator disassembly case, 11 disassembly schemes are generated, accounting for task failures. The results indicate that, compared to traditional disassembly methods, the rebalancing approach not only optimizes the station loads but also increases revenue by 11.98 %, demonstrating the effectiveness of the proposed model and method in handling task failures on disassembly lines.

1. Introduction

The rapid development of technology has accelerated the pace of product updates and shortened the service life of products, resulting in a large amount of e-waste. The Global E-Waste Monitor 2024 reports that global e-waste generation increased by 82 % from 2010 to 2022, reaching a record 62 million metric tons, with only 22.3 % of resources being adequately recycled. This represents a waste of valuable metals worth \$91 billion [1]. E-waste contains numerous recyclable resources and hazardous substances [2]. Their disassembly and recycling not only improve resource utilization but also reduce environmental pollution, offering significant environmental and economic benefits [3], as shown

in Fig. 1(a). A streamlined disassembly process enhances automation and disassembly efficiency [4]. The disassembly line balancing (DLB) problem is a critical planning issue for disassembly lines. It involves assigning a set of disassembly tasks to workstations under various constraints while optimizing predetermined performance indicators [5].

The complexity and unknown state of e-waste mean that the disassembly process is not static [6]. One difficulty of DLB problems is the need to flexibly handle dynamic disassembly problems and complex disassembly tasks [7]. As shown in Fig. 1(b), task failures during disassembly can disrupt the entire process, reducing disassembly rates and resource recovery efficiency [8]. Reallocating remaining disassembly tasks after a task failure and realizing the disassembly line

* Corresponding author.

E-mail address: luoyabo1973@163.com (Y. Luo).

<https://doi.org/10.1016/j.jmansys.2025.02.009>

Received 5 January 2025; Received in revised form 11 February 2025; Accepted 17 February 2025

Available online 26 February 2025

0278-6125/© 2025 The Society of Manufacturing Engineers. Published by Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

rebalancing can effectively reduce losses. Therefore, a segmented model for disassembly line balancing and rebalancing considering task failures is essential to recover losses and achieve an effective disassembly scheme.

In addition, given that this issue is a complex combinatorial optimization problem with dynamic disturbances, the optimization process is very complex. It is crucial to further design an effective optimization algorithm to obtain high-quality disassembly schemes. In recent years, the combination of reinforcement learning algorithms (such as Q-learning) and meta-heuristic algorithms has attracted the attention of researchers [9,10]. This method can flexibly adjust the algorithm dynamically according to the actual optimization situation [11]. At present, in combinatorial optimization problems, Q-learning has been introduced into various meta-heuristics, and demonstrates significant effectiveness in accelerating convergence and improving solution quality [12]. These studies differ in the selection of algorithms and the action mode of Q-learning. Based on this, this paper combines genetic algorithm with Q-learning according to the characteristics of the problem, and guides the selection of evolutionary strategy through the change of population performance in the iteration process, to effectively solve the problem.

Based on the above status and analysis, the main contributions of this article are as follows:

- (1) A mathematical model for disassembly line balancing and rebalancing problems considering task failures (DLBRP-TF) is proposed.
- (2) An adaptive multi-objective genetic algorithm based on Q-learning (AMOGA-QL) is proposed, including four state features and six actions to enhance the dynamic optimization capability.
- (3) The proposed model and algorithm are validated through test cases and applied to an actual refrigerator disassembly case.
- (4) This study assists decision-makers in flexibly choosing disassembly schemes based on actual needs.

The remainder of this study is organized as follows. Section 2 reviews related literature. Section 3 constructs the mathematical model of DLBRP-TF. Section 4 introduces the proposed AMOGA-QL. Section 5 presents the performance validation of the proposed algorithm. Section 6 discusses a practical refrigerator disassembly case. Section 7 concludes the study.

2. Literature review

The research on disassembly line balancing is mainly analyzed from the aspects of problem type, optimization objective, and optimization algorithm, and then the gaps of existing research, and the research motivation of this study are presented. The comprehensive analysis of the literature is shown in Table 1.

2.1. Problem type

The disassembly process is inherently complex due to several uncertain factors [33]. The presence of these factors increases the risk of task failures, and the precedence relationships among tasks in the disassembly line could prevent subsequent tasks from proceeding as planned. Current research on disassembly line balancing under task failure primarily focuses on partial disassembly [27,34]. These studies emphasize preventive disassembly sequences to minimize expected losses in revenue [35] or carbon emissions [23], and do not consider the value of the remaining tasks. However, it is significant to reallocate the remaining tasks after failures to rebalance the line. This process can restore part of the disassembly line's value.

Despite the significance of rebalancing after task failures, research in this area remains limited. Laili, et al. [36] explored flexible adjustments following automated operation failures, and used backup operations to enable online recovery. This method alters the disassembly actions to allow the failed task to proceed, essentially repairing the original sequence but failing to address more complex disassembly failures. Altekin, et al. [13] proposed a mixed-integer-programming based on the predictive-reactive approach to find the best backup operation for every possible failure by rebalancing. The results show that the monetary throughput lost due to task failure can be recovered by rebalancing. Based on this, Singh, et al. [26] developed a recursive approach to find the best corrective action for each potential failure. However, their approaches to disassembly line recovery rely on a fixed sequence, neglecting the interdependence of sequences before and after failures. Thus, a comprehensive evaluation of disassembly schemes before and after failures, selecting the most effective combination, can enhance disassembly line efficiency and assist decision-makers in developing more practical disassembly strategies.

2.2. Optimization objectives

In disassembly line balancing, the number of workstations and the smoothness index are the two fundamental optimization objectives [17]. The number of workstations affects construction costs, while the smoothness index reflects workload distribution and line efficiency when the perfect balance is unattainable [2]. Recently, green manufacturing has gained attention, and energy conservation and emission reduction have become integral to disassembly line issues. Energy consumption arises from both the disassembly process and equipment transportation [20]. Considering energy consumption's impact on DLB problems can not only reduce environmental effects but also improve economic benefits [25]. In the DLBRP-TF, the cost of rebalancing is challenging to measure directly. Most studies treat rebalancing as an indirect balancing problem under task allocation constraints, without explicitly considering adjustment costs [14]. Rebalancing often entails equipment stoppages and tool changes. To save resources and reduce costs effectively, it is generally desirable for

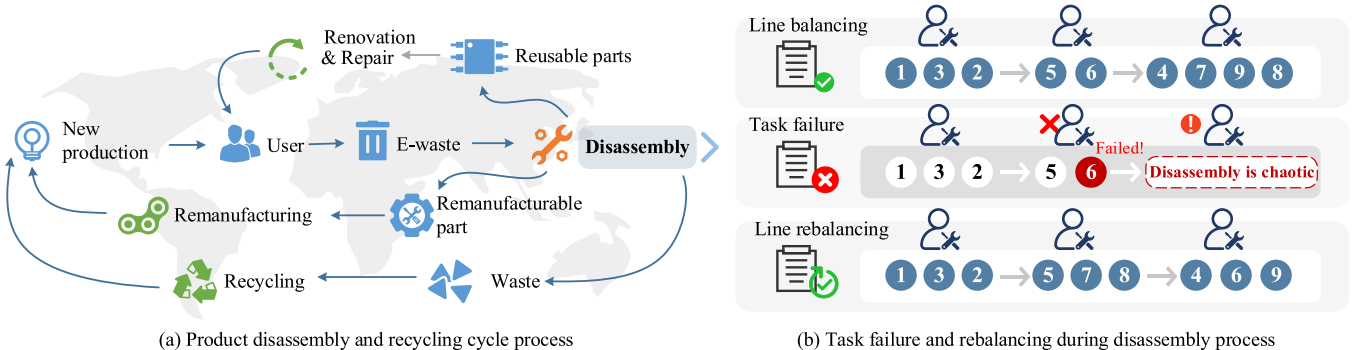


Fig. 1. Schematic diagram of e-waste recovery and disassembly.

Table 1
Literature analysis.

Reference	Problem type			Objective					Algorithm			
	DLB	LRB	DLB + LRB	NW	SI	AS	DR	EC	EM	HM	MHM	QL
[2]	✓			✓	✓					✓		
[3]	✓				✓		✓				GA	
[10]	✓										PSO	✓
[13]		✓					✓		✓			
[14]		✓		✓	✓	✓					GA	
[15]		✓				✓			✓			
[16]	✓						✓				ACO	
[17]	✓			✓	✓		✓			✓		
[18]		✓					✓		✓			
[19]	✓						✓				PSO	
[20]	✓			✓	✓		✓	✓			FPA	
[21]	✓						✓	✓			FOA	
[22]	✓			✓	✓		✓			✓		
[23]	✓						✓	✓			HGWO	
[24]	✓						✓			✓		
[25]	✓				✓		✓				HWCA	
[26]	✓						✓		✓			
[27]	✓						✓			✓		
[28]	✓					✓	✓					
[29]	✓						✓	✓	✓			
[30]	✓				✓		✓		✓			✓
[31]	✓						✓		✓			✓
[32]	✓						✓					
The present study			✓	✓	✓	✓	✓	✓			GA	✓

DLB: disassembly line balancing; LRB: line rebalancing; NW: number of workstations; SI: smoothness index; AS: adjustments of sequences; DR: disassembly revenue; EC: energy consumption; EM: exact method; HM: heuristic method; MHM: meta-heuristic method; GA: genetic algorithm; PSO: particle swarm optimization; ABC: artificial bee colony; ACO: ant colony optimization; FPA: flower pollination algorithm; FOA: fruit fly optimization algorithm; HGWO: hybrid grey wolf optimizer; HWCA: hybrid water cycle algorithm; QL: Q-learning.

the rebalanced sequence to closely resemble the original one [15]. Therefore, the movement of tasks between workstations before and after rebalancing can be used as an optimization objective to simplify the problem. To solve DLBRP-TF, it is essential to consider the number of workstations, smoothness index, adjustments in the rebalanced sequence, and energy consumption. Additionally, energy consumption should be used to establish connections among different disassembly scenarios, enabling a thorough and rational evaluation of the disassembly schemes.

2.3. Optimization algorithm

The DLB is an NP-hard problem [37] and is typically solved using exact algorithms [18], heuristic algorithms [38], and meta-heuristic algorithms [32]. Meta-heuristics, such as genetic algorithm (GA) [22], artificial bee colony [28], and particle swarm optimization [19], are particularly popular for solving multi-objective problems in static environments. Among them, PSO tends to converge quickly, but it may fall into local optimization under complex constraints. ABC is suitable for the case where the local solution is better, but the global search ability is relatively weak. GA has strong global search ability, adaptability, and flexibility in solving the DLB, especially in the context of complex constraints and multi-objective optimization [39]. Therefore, genetic algorithm is suitable for solving DLBRP-TF.

For complex problems with uncertainties, obtaining high-quality solutions efficiently is challenging with simple genetic algorithms. Algorithm improvements are necessary to achieve better performance. Recently, reinforcement learning, especially Q-learning, has gained attention for its adaptability and ability to avoid local optima in dynamic and uncertain environments [40]. Q-learning continuously optimizes decisions by learning from historical and current data [41]. At present, the combination of Q-learning with meta-heuristics has proven effective in combinatorial optimization [42,43]. Some scholars have applied this method to the disassembly line balancing problem. Chu, et al. [10] designed a hybrid particle swarm optimization with the Q-learning algorithm to select the most suitable optimizer in the

evolutionary process. Ren, et al. [30] improved artificial bee colony algorithms are developed to solve the bi-objective disassembly line scheduling problem, and designed two Q-learning-based strategies for selecting high-quality local search operators. Rahman, et al. [44] introduced Q-learning into genetic algorithm to automatically select which layer of chromosome to activate the crossover or mutation action based on the diversity and convergence of the algorithm. These studies have proved the role of the introduction of Q-learning in DLBPs. In practical application, it is also necessary to select the specific algorithms and action modes of Q-learning according to the characteristics of the problem and model. For the large-scale complex problem with dynamic disturbance proposed in this paper, the combination of Q-learning and genetic algorithm can significantly improve the efficiency of the algorithm and optimize the multi-objective disassembly line balancing.

3. Problem statement

3.1. Problem description

This study proposes a disassembly line rebalancing model considering task failures. During disassembly, hazardous tasks, which have a probability of failure, can disrupt the initial balance. Normally, the disassembly line follows a planned sequence (Fig. 2(a)). When a task fails, the workstation where the failed task is located may exceed the cycle time, and the subsequent disassembly tasks are unable to proceed normally (Fig. 2(b)). The disassembly line can be rebalanced by re-allocating subsequent tasks (Fig. 2(c)). To swiftly restore balance and maintain profitability in the event of task failure, contingency plans can be devised for different failure scenarios. Since multiple optimal solutions may exist, the disassembly scheme is not unique.

As illustrated in Fig. 3, there are nine disassembly tasks, with tasks 2 and 7 identified as hazardous. During disassembly, four possible scenarios may arise:

- 1) In scenario 1, all disassembly tasks proceed successfully. One sequence [1–9] is selected as an example.

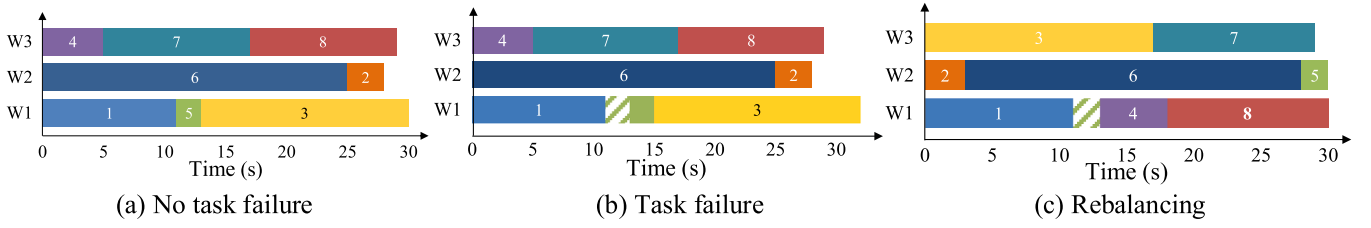


Fig. 2. Diagram of disassembly scheme considering task failure.

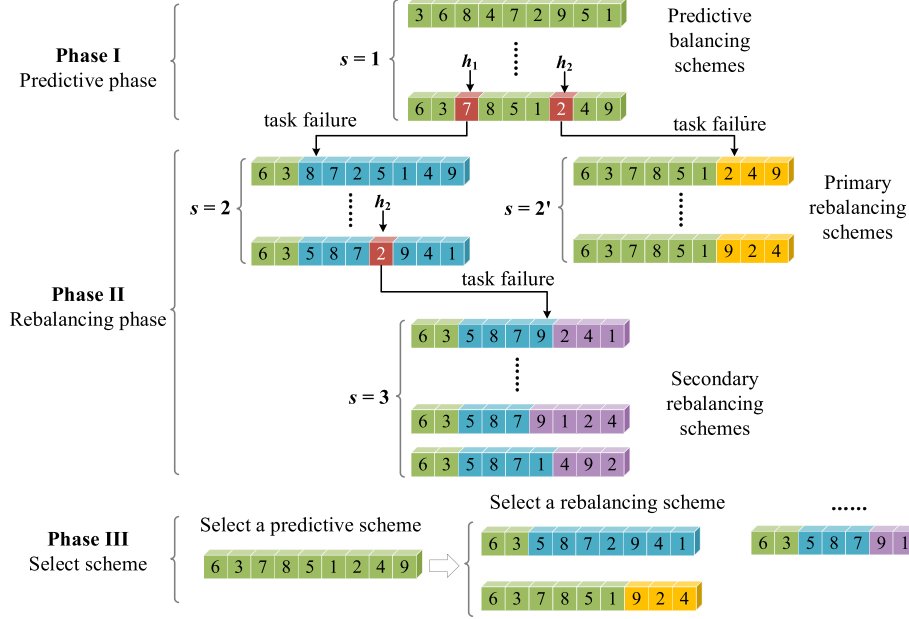


Fig. 3. Diagram of disassembly scheme considering task failure.

- 2) In scenario 2, task 7 fails. The completed sequence [3,6] remains unchanged, and the remaining tasks are reallocated.
- 3) Similarly, in scenario 2', task 2 fails. The sequence [3,5–8] remains unchanged, and the remaining tasks are reallocated.
- 4) In scenario 3, both hazardous tasks fail. The sequence is reallocated based on the failure order in scenario 1, as shown in the sequence [1–9] when $s = 3$. Here, the sequence [3,6] belongs to the predicted disassembly sequence, [5,7,8] to the first rebalanced sequence, and [1,2,4,9] to the second rebalanced sequence.

Each scenario can generate multiple disassembly sequences, leading to thousands of possible combinations. The comprehensive energy consumption is calculated based on the probability of each scenario, and the best combination is selected according to the optimization objectives.

The DLBRP-TF involves three subproblems: 1) the balancing problem of the predicted sequence (without considering task failure); 2) the rebalancing problem of new sequences driven by task failure; and 3) the scheme selection problem under comprehensive scenarios. To devise a solution strategy for the DLBRP-TF, the following assumptions are proposed:

- The failure probabilities of hazardous tasks are known and can be determined based on historical data.
- Each hazardous task can experience at most one failure during disassembly.
- Task failures occur at the final step of the disassembly action, occupying the same time as successful disassembly, and all

subsequent workstations can immediately adjust their work status when a failure occurs.

3.2. DLBRP-TF model

The mathematical model established is as follows, and the parameters and variables required by the model are as follows:

Indices:

m	Index of workstation.
n	Index of disassembly task.
v, w	Workstation index, $v, w \in M$.
i, j	Task index, $i, j \in N$.
M	Workstations set, $m \in M$.
N	Disassembly task set, $n \in N$.
T_c	The cycle time of the disassembly line.
P_1	Load power during operation.
P_2	Standby power.
P_3	Operating power of the workstation.
t_n	The disassembly time of task n .
h_1, h_2	Index of hazardous task.
H	Set of hazardous tasks, $h_1, h_2 \in H$.
M_r	Set of workstations opened for rebalancing sequence.
M_a	Set of total workstations opened after rebalancing.
N_0	Set of disassembly tasks completed before a task failure.
f_n	Probability of failure of hazardous task n .
s	Index of scenario, $s \in \{1, 2, \dots, 2^{ H }\}$.
p_s	Probability of occurrence of scenario.
e	Comprehensive energy consumption of the sequence under various scenarios.
e_s	Energy consumption under scenario s .

(continued on next page)

(continued)

Indices:	
$T_{op, s}$	Working time of the disassembly line under scenario s , which is the total time for performing tasks.
$T_{st, s}$	Standby time of the disassembly line under scenario s , which is the total idle time of workstations.
Decision variables:	
d_m	The workstation open state: $d_m = 1$ if workstation m is open; otherwise $d_m = 0$.
T_m	The working time in workstation m , which is a non-negative variable.
x_{mn}	Task assignment variable: if task n is assigned to workstation m , $x_{mn} = 1$; otherwise $x_{mn} = 0$.
y_{ij}	The precedence relationship between tasks: if task i is the immediate predecessor of task j , $y_{ij} = 1$; otherwise $y_{ij} = 0$.
l_1, l_2	Index of rebalance occurrence, if h_1 precedes h_2 in the predicted sequence position, $l_1 = h_1, l_2 = h_2$; otherwise $l_1 = h_2, l_2 = h_1$.
s_n^0	Index of the workstation where task n is located in the predictive sequence.
s_n	Index of the workstation where task n is located in the rebalanced sequence.
b_0	Penalty multiplier for workstation changes, which is a non-negative variable.
v_s	Adjustments in the rebalanced sequence under scenario s .

3.2.1. Predictive disassembly line balancing model

This section discusses the DLB problem model under the assumption of no task failure. The optimization objective of this section includes minimizing the number of workstations, smoothness index, and energy consumption of the disassembly line. The expression for these objectives is:

$$\min F = (f_1, f_2, f_3) \quad (1)$$

Sub-objective 1: The number of workstations. When the cycle time is fixed, increasing the number of workstations leads to more operating disassembly equipment, raising both disassembly costs and energy consumption.

$$\min f_1 = |M| \quad (2)$$

$$|M| = \sum_{m \in M} d_m \quad (3)$$

Sub-objective 2: The smoothness index. The smoothness index reflects the relationship between idle time and disassembly line efficiency. Greater variability in idle times across workstations indicates imbalances in task allocation, reducing the overall efficiency.

$$\min f_2 = \sum_{m \in M} (T_c - T_m)^2 \quad (4)$$

$$T_m = \sum_{n \in N} x_{mn} t_n \quad \forall m \in M \quad (5)$$

Sub-objective 3: Disassembly line energy consumption. The energy consumption of a disassembly line arises from disassembly energy, standby energy, and workstation operational energy, all of which are closely linked to the working and idle times of the disassembly line.

$$\min f_3 = P_1 \sum_{m \in M} T_m + P_2 \sum_{m \in M} (T_c - T_m) + |M| T_c P_3 \quad (6)$$

The constraint conditions are as follows:

Constraint 1: Constraint on the number of workstations. When the cycle time of the disassembly production line is fixed, the minimum number of actually opened workstations is $\lceil \sum_{n \in N} t_n / T_c \rceil$. In this scenario, each workstation operates at full capacity without any idle time.

$$\left\lceil \sum_{n \in N} t_n / T_c \right\rceil \leq |M| \leq |N| \quad (7)$$

Constraint 2: Constraint on workstation working time. The total operation time of all disassembly tasks within a workstation does not exceed the workstation's cycle time.

$$T_m \leq T_c \quad \forall m \in M \quad (8)$$

Constraint 3: Each disassembly task must be completed, and it can only be assigned to one workstation.

$$\sum_{m \in M} x_{mn} = 1 \quad \forall n \in N \quad (9)$$

Constraint 4: Precedence constraint. The operation can only be assigned after all its predecessors have been assigned.

$$\sum_{v \in M} v \cdot x_{vi} \leq \sum_{w \in M} w \cdot x_{wj}, \quad \forall i, j \in \{i, j | y_{ij} = 1\} \quad (10)$$

3.2.2. Task-failure-driven disassembly line rebalancing model

During disassembly, task failures disrupt the original balance, and it is impossible to proceed with subsequent tasks as planned. To enhance the disassembly efficiency of subsequent tasks and improve the overall effectiveness of the disassembly line, it is necessary to reallocate tasks that have not been disassembled and seek the optimal solution that meets objectives and constraints. In the event of multiple task failures, they are to be reallocated sequentially based on the order of their occurrence. Each reassignment is to build upon the previous rebalanced sequence to achieve the next round of rebalancing.

A model is established using the first rebalancing occurrence as an example. The comprehensive optimization objectives of this section include minimizing the number of workstations opened during line rebalancing, smoothness index, adjustments in the rebalanced sequence, and disassembly line energy consumption. The expression for these objectives is:

$$\min Z = (z_1, z_2, z_3, z_4) \quad (11)$$

Sub-objective 1: Number of workstations opened during line rebalancing.

$$\min z_1 = |M_a| \quad (12)$$

$$|M_a| = s_{l_1}^0 + \sum_{m \in M_r} d_m - 1 \quad (13)$$

Sub-objective 2: The smoothness index. The smaller the smoothness index, the better the balancing performance of the rebalanced sequence.

$$\min z_2 = \sum_{m \in M_r} (T_c - T_m)^2 \quad (14)$$

Sub-objective 3: Adjustments in the rebalanced sequence. Rebalancing the disassembly sequence can cause changes such as equipment startup, shutdown, and tool replacement, leading to costs that are difficult to measure directly. To simplify the problem, the adjustments of the rebalanced sequence (i.e., the movement of tasks between workstations before and after rebalancing) are used.

$$\min z_3 = \sum_{n \in N - N_0} |s_n - s_n^0| \quad (15)$$

Sub-objective 4: Disassembly line energy consumption. Compared to the predictive phase, the energy consumption on the disassembly line during rebalancing also includes the energy generated by adjusting the sequence. A penalty multiplier b_0 is introduced to quantify the additional energy consumption caused by these adjustments.

$$\min z_4 = P_1 \sum_{m \in M_r} T_m + P_2 \sum_{m \in M_r} (T_c - T_m) + |M_r| T_c P_3 + b_0 z_3 \quad (16)$$

Constraints (17)–(20) have the same meaning as in Section 3.2.1.

$$\left\lceil \sum_{n \in N - N_0} t_n / T_c \right\rceil \leq |M_r| \leq |N - N_0| \quad (17)$$

$$T_m \leq T_c \quad \forall m \in M_r \quad (18)$$

$$\sum_{m \in M_r} x_{mn} = 1 \quad \forall n \in N - N_0 \quad (19)$$

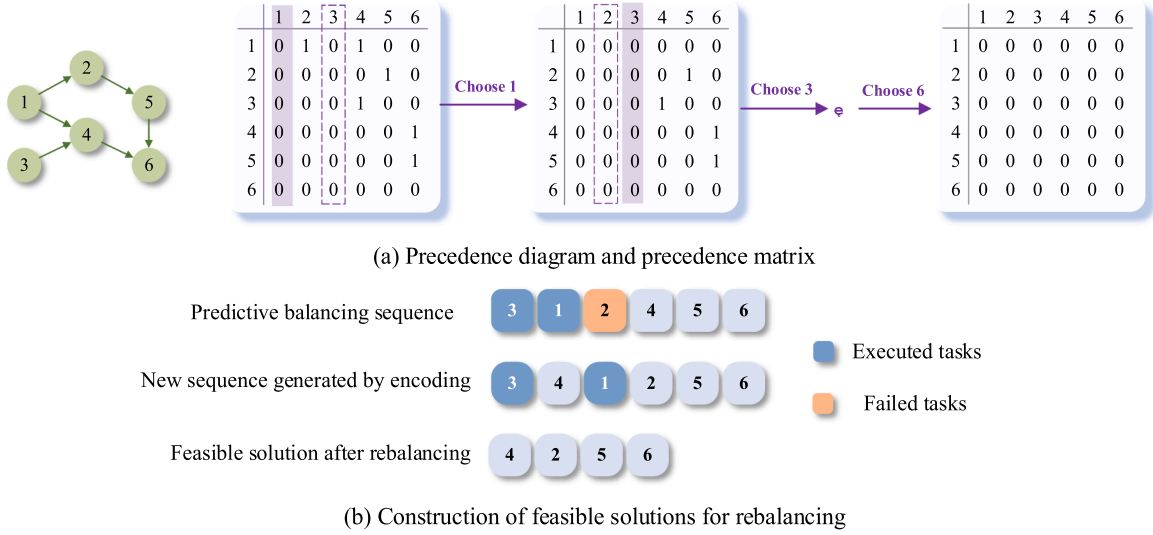


Fig. 4. Encoding strategy.

$$\sum_{v \in M_r} v \cdot x_{vi} \leq \sum_{w \in M_r} w \cdot x_{wj}, \quad \forall i, j \in \{i, j | y_{ij} = 1\} \quad (20)$$

3.2.3. Selection of the combined solutions

In typical scenarios, disassembly tasks follow a planned sequence, but hazardous tasks can disrupt operations. Each hazardous task has two potential outcomes: successful completion (probability is $1 - f_n$) or failure (probability is f_n). Thus, there are $2^{|H|}$ possible scenarios for the disassembly sequence. This study primarily explores the case where $|H| = 2$. For two hazardous tasks h_1 and h_2 , this results in $2^2 = 4$ scenarios. The following lists these four scenarios and their probabilities.

Scenario 1: Both hazardous tasks are completed. $p_1 = (1 - f_{h1})(1 -$

$f_{h2})$.

Scenario 2: Task h_1 fails, and task h_2 succeeds. $p_2 = f_{h1}(1 - f_{h2})$.

Scenario 3: Task h_1 succeeds, and task h_2 fails. $p_3 = (1 - f_{h1})f_{h2}$.

Scenario 4: Both hazardous tasks fail. $p_4 = f_{h1}f_{h2}$.

The disassembly process consumes a certain amount of energy. Since the disassembly tasks on the line follow a specific plan, the actual energy consumption is influenced by the contingency plans for various scenarios. When selecting a disassembly plan, it is essential to consider both normal execution and the potential failure of hazardous tasks and make the comprehensive energy consumption under various scenarios as low as possible on the premise of meeting the objectives required for each phase.

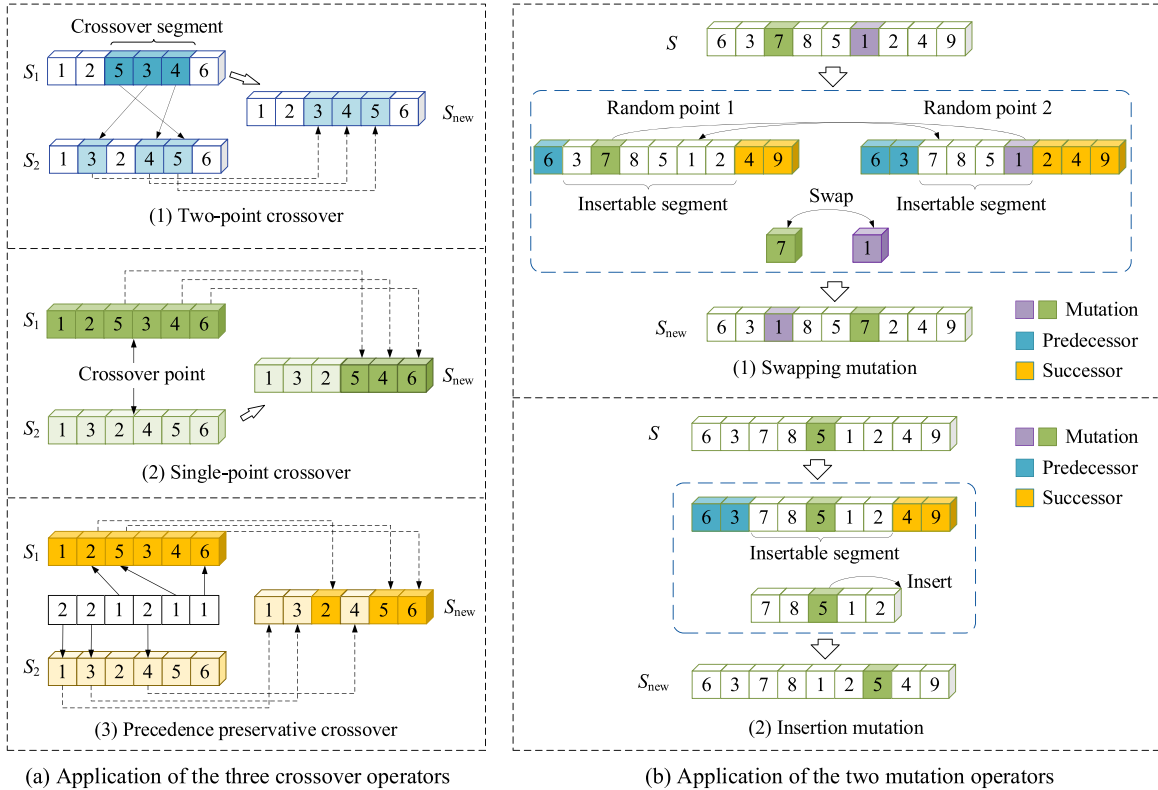


Fig. 5. The three crossover operators and mutation operators included in the AMOGA-QL.

Energy consumption includes task execution, standby, workstation operation, and rebalancing energy. The comprehensive energy consumption is calculated by summing energy across scenarios weighted by their probabilities. According to the probability of each scenario, the comprehensive energy consumption of the disassembly line is calculated as:

$$e = \sum_{s=1}^{2^H} p_s e_s \quad (21)$$

where

$$e_s = T_{op,s}P_1 + T_{st,s}P_2 + (T_{op,s} + T_{st,s})P_3 + b_0v_s \quad (22)$$

The nested nature of the DLBRP-TF model increases solution complexity, requiring high-quality solutions to balance upper- and lower-level objectives. Traditional methods struggle to provide efficient solutions, highlighting the need for advanced, tailored algorithms.

4. The adaptive multi-objective genetic algorithm based on Q-learning

The combination of meta-heuristic algorithms and Q-learning offers strong global search capabilities and better adaptability for complex problems, making it ideal for optimizing disassembly line balancing [45]. This study introduces an adaptive multi-objective genetic algorithm based on Q-learning (AMOGA-QL) to solve the problem. The introduction of Q-learning enables dynamic adjustments based on runtime conditions, expanding the search scope and accelerating algorithm convergence. The algorithm includes encoding, decoding, learning, and evolutionary operations. The genetic algorithm performs global searches through crossover and mutation, while Q-learning optimizes the evolutionary process to rapidly find high-quality solutions.

4.1. Encoding based on precedence relationships

The encoding is constructed to generate feasible disassembly sequences based on the characteristics of the disassembly line. Random encoding can lead to infeasible solutions due to constraints, adversely affecting algorithm efficiency. Therefore, this study adopts an integer permutation encoding based on a precedence matrix, to enable the algorithm to eliminate infeasible solutions in the initial stage and improve the performance of the initial solutions. Firstly, a precedence relationship diagram is drawn based on the constraints among tasks. Next, a precedence matrix $Y = [y_{ij}]_{n \times n}$ is established based on the precedence diagram. The dimension n of the matrix represents the number of tasks. The steps to construct the disassembly sequence are as follows:

Step 1: An unassigned task i with columns of 0 (indicating all precedent tasks have been assigned) is randomly selected from the matrix Y among the candidate tasks and is assigned to the current position k of the sequence.

Step 2: Task i is removed from the candidate tasks, and matrix Y is updated to release the constraints associated with task i , that is, $y_{ji} = 0$, and $y_{ji} = 0, \forall i \in \{1, 2, \dots, n\}$.

Step 3: The task is assigned to the next position, $k \leftarrow k + 1$.

Step 4: Steps 1–4 are repeated until all parts are dispensed and the disassembly sequence is obtained.

As shown in Fig. 4(a), the sequence is divided into n segments, each representing a task, to construct a feasible solution for the disassembly sequence. Through this encoding method, each task's precedent task has already been assigned when it is assigned, ensuring that the final encoding sequence conforms to the precedence relationship constraint.

After constructing feasible solutions, for the rebalancing of partial sequences following task failure, it's necessary to exclude tasks that have already been completed. As shown in Fig. 4(b), for example, in the original balanced sequence [1–6], a failure occurs at task 2. Tasks 3 and 1 before task 2 have already been completed. Therefore, the remaining tasks [2,4–6] are re-encoded to obtain a feasible solution.

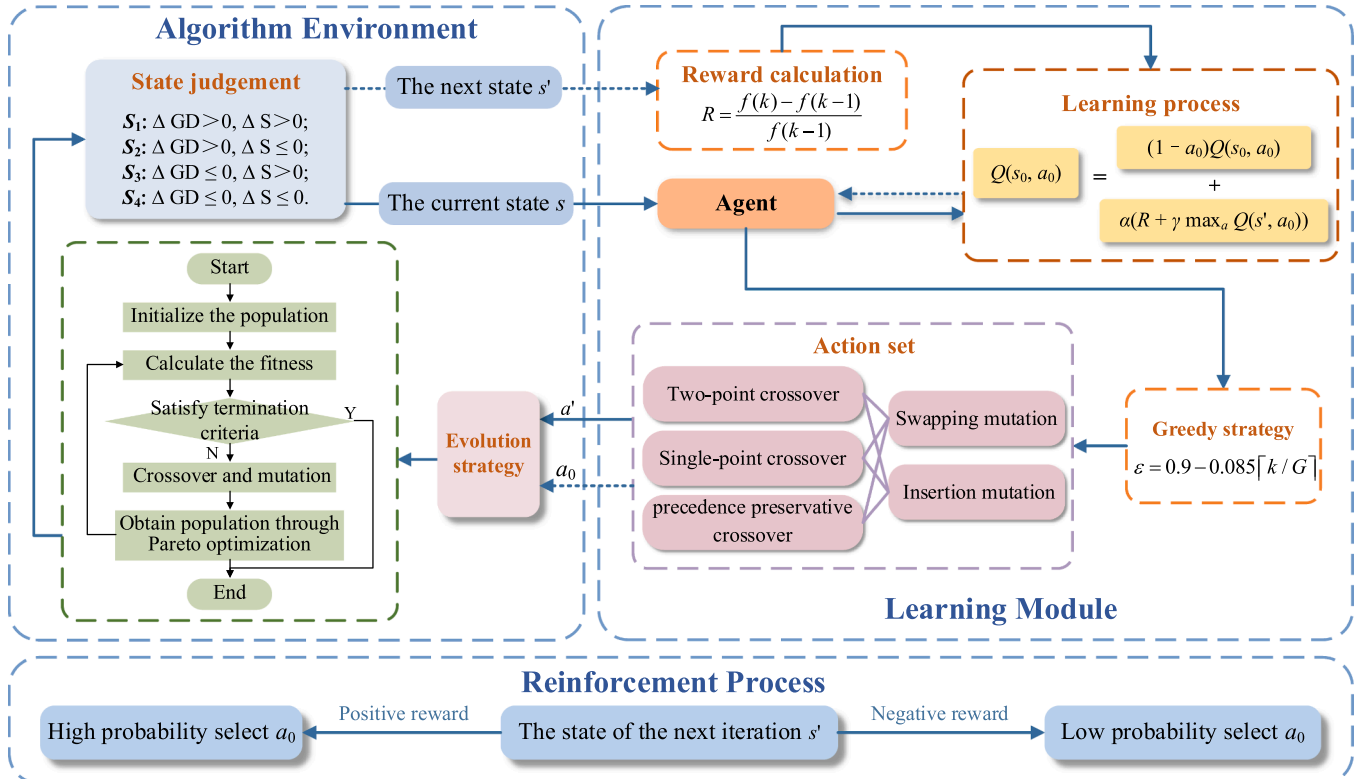


Fig. 6. The framework of the Q-learning.

4.2. Decoding strategy

The encoding scheme alone cannot evaluate the quality of individuals, requiring further decoding to allocate disassembly tasks to workstations and form a complete scheme. During decoding, it's crucial to ensure that the operation time for tasks in each workstation does not exceed its cycle time.

For the predictive phase, during decoding, tasks are allocated into workstations in the order of feasible solutions. When a workstation's remaining idle time is insufficient to complete the next disassembly task, a new workstation is opened. This ensures minimal activation of workstations and reduces their idle time. In the rebalancing phase, the failed task's workstation and position are identified, and task allocation resumes from the point of failure at that workstation, as shown in Algorithm 1.

Algorithm 1. Decoding the sequence in the rebalancing phase.

Input: The encoded sequence for rebalancing phase X , The remaining number of remaining tasks $|N_r|$, task durations t_i , cycle time T_c , the workstation of the failed task m_0 , the workstation's elapsed time T_0 .

Output: Disassembly scheme.

```

 $m = 1$ ; / Start workstation count /
 $T_m = T_0$ ; / Start timing /
for  $p = 1$  to  $|N_r|$  do /  $p$  corresponds to the position in the disassembly sequence /
    Locate task  $i$  at position  $p$  in the disassembly sequence.
    if  $(T_c - T_m) \geq t_i$  then / The workstation has ample time left to complete the disassembly task  $i$  /
         $T_m = T_m + t_i$ ; / The workstation's working time after assigning task  $i$  /
         $W_i = m + m_0 - 1$ ; / Record the workstation where task  $i$  is located /
    else / The workstation has inadequate time left to complete the disassembly task  $i$  /
         $m = m + 1$ ; / Open a new workstation /
         $T_m = 0$ ; / Reset the workstation's working time /
         $T_m = t_i$ ; / Update the workstation's working time /
         $W_i = m + m_0 - 1$ ; / Record the workstation where task  $i$  is located /
    end if
end for

```

4.3. Crossover operation and mutation operation

Evolutionary operations are used to expand the search range and accelerate the convergence speed of algorithms. This study includes three crossover operations and two mutation operations.

To broaden the search range and ensure a thorough exploration of the solution space during the iterative optimization process, it is necessary to generate new feasible solutions. Three types of crossover operators are employed to update the population. The detailed crossover process is illustrated in Fig. 5(a).

- (1) Two-point crossover: Two individuals are randomly selected from the population as parents, and a gene segment is randomly chosen from both parents to serve as the crossover segment. The crossover is performed using a mapping method, where the genes within the crossover segment are reallocated according to the corresponding gene sequence in the other parent.
- (2) Single-point crossover: Two individuals are randomly selected from the population as parents. A crossover point is randomly chosen, and the sequence after this point is used as the crossover

segment, while the sequence before the point remains unchanged. The crossover is performed using a mapping method, where the genes within the crossover segment are reallocated according to the corresponding gene sequence in the other parent.

- (3) Precedence preservative crossover: A random execution sequence is generated, and the genes from both parents are preserved in order. The genes from the selected parent are chosen according to the random sequence, and any genes that have already appeared in the offspring are removed during the process.

Random mutations can produce infeasible solutions. Two mutation strategies based on precedence relationships are adopted. The detailed mutation process is illustrated in Fig. 5(b).

- (1) Swapping mutation: Two points in the sequence are randomly

selected as mutation points. Their insertable positions are determined based on precedence constraints, and if they can be swapped within these constraints, a new individual is generated.

- (2) Insertion mutation: A mutation point is randomly selected from an individual and determines the closest preceding and successor tasks. The sequence between these tasks defines the insertable segment. If the chosen mutation point has no valid insertable segment, a new mutation point is selected. Randomly select a position in the insertable segment for insertion to obtain a new individual.

4.4. Population update strategy

The DLB problem is a typical multi-objective optimization problem, where the optimization of multiple objectives is not simultaneous. To ensure the balance of the solution set, the Pareto non-dominated solution is used to filter the objectives. This method compares feasible solutions based on dominance relationships in multi-objective problems and selects the non-dominated set, known as the Pareto front [46]. Solutions in the Pareto front are not dominated by each other, and each

emphasizes different objectives. Decision-makers can choose solutions based on specific needs. During population updates, Pareto front solutions are added to retain optimal solutions and accelerate convergence.

4.5. Q-learning strategy

Q-learning selects the best strategy in each iteration by evaluating the improvement of each strategy using historical data. It chooses the action with the highest Q-value in the current state to optimize performance. The key components of Q-learning are the state, action, Q-value table, and reward function.

4.5.1. Actions and state settings

During the execution of the algorithm, the Pareto front reflects the solving ability of the algorithm. The algorithm aims to find superior solutions; therefore, the state setting is related to the evaluation of the quality of multi-objective solution sets. Two multi-objective evaluation indicators are introduced: Generational Distance (GD) [47] and Spacing [48]. Since the true Pareto front is unknown, the previous generation's Pareto front is used as the reference for evaluating performance. During evolution, the values of GD and Spacing will fluctuate, and the changes are denoted as ΔGD and ΔS . The positive or negative values of these changes reflect the population's convergence and uniformity speed. The four combinations are: 1) $\Delta GD > 0, \Delta S > 0$; 2) $\Delta GD > 0, \Delta S \leq 0$; 3) $\Delta GD \leq 0, \Delta S > 0$; and 4) $\Delta GD \leq 0, \Delta S \leq 0$. These four scenarios are defined as the four states for Q-learning.

By using Q-learning, the population can be guided to choose the best evolutionary strategies and enhance the search capability of the algorithm. The evolutionary process involves combining the two crossover operators and the two mutation operators from Section 4.3 into six actions: 1) Two-point crossover & Swapping mutation; 2) Two-point crossover & Insertion mutation; 3) Single-point crossover & Swapping mutation; 4) Single-point crossover & Insertion mutation; 5) Precedence preservative crossover & Swapping mutation; and 6) Precedence preservative crossover & Insertion mutation.

4.5.2. Reward definition

The performance of selected actions is evaluated based on reward values, reflected in the population's fitness. If the current population's fitness exceeds that of the previous generation, the reward value is positive; otherwise, it is negative. The reward value is calculated as follows:

$$\text{Reward} = (f(k) - f(k-1)) / f(k-1) \quad (23)$$

4.5.3. Improved greedy strategy

ϵ -greedy is a commonly used exploration strategy. When making a decision, an action is randomly selected with a probability of ϵ , and the action with the highest known Q-value is selected with probability $1 - \epsilon$. When $\epsilon \geq r_0$, the action corresponding to the maximum Q-value is chosen. When $\epsilon < r_0$, a random action is selected. The formula is:

$$\pi(s_t, a_t) = \begin{cases} \arg \max_a (Q(s_t, a_t)), & \epsilon \geq r_0 \\ a(\text{randomly}), & \epsilon < r_0 \end{cases} \quad (24)$$

In the classical ϵ -greedy strategy, ϵ is a fixed value and can limit the adaptability of Q-learning. A discount factor γ is introduced to automatically adjust the value of ϵ . Initially, the value of ϵ is set to a maximum value to enhance the algorithm's ability to explore new solutions. As the algorithm progresses, the value of γ gradually increases while ϵ decreases, and the learning outcomes in the later stages are retained. The calculation formulas are as follows:

$$\epsilon = 0.9 - 0.85\gamma \quad (25)$$

$$\gamma = \delta[k/G] \quad (26)$$

where δ is a fixed value, k represents the current iteration number, and G represents the maximum number of iterations.

4.5.4. Q-learning process

In Q-learning, there are four states and six actions. The dimension of the Q-value table is 4×6 , and the initial value is set to 0. The Q-value update formula during the algorithm's execution is:

$$Q(s_0, a_0) = (1 - \alpha) \cdot Q(s_0, a_0) + \alpha \left(R + \gamma \cdot \max_{a'} Q(s', a_0) \right) \quad (27)$$

where $Q(s_0, a_0)$ represents the Q-value that can be obtained by taking action a_0 in the current state s_0 ; R represents the reward value obtained by taking action a_0 in state s_0 ; $\alpha \in (0, 1)$ is the reinforcement learning coefficient; $\gamma > 0$ is the discount factor; and $\max_{a'} Q(s', a_0)$ represents the maximum Q-value that can be obtained by taking action a' in the next state s' . The Q-learning process is shown in Fig. 6.

4.6. Algorithm steps

Step 1: The parameters are initialized: population size N , maximum number of iterations G , crossover probability P_c , mutation probability P_m ; the Q-value table is set to 0 at the beginning.

Step 2: The population is initialized based on the encoding scheme and population size. Tasks are assigned to workstations following the decoding scheme. The objective function values of the initial population are then calculated.

Step 3: The current iteration number is set to 1, $t = 0$.

Step 4: Calculate the population fitness.

Step 5: When $t = 0$, a random action is selected and the state is set to State 4) $\Delta GD \leq 0, \Delta S \leq 0$. When $t > 0$, the reward value R and the Q-value table are updated.

Step 6: A random number $Rand$ is generated. If $Rand < P_c$, a crossover action is selected based on the Q-value table.

Step 7: A random number $Rand$ is generated. If $Rand < P_m$, a mutation action is selected based on the Q-value table. The crossover and mutated population are combined with the initial population.

Step 8: The Pareto front is calculated and stored in the new population. The population size is then updated and maintained accordingly.

Step 9: $t = t + 1$.

Step 10: If the maximum number of iterations is reached, the loop is terminated. Otherwise, return to Step 4.

Step 11: The final Pareto front serves as the candidate scheme for the predictive phase. Each individual is rebalanced across various scenarios following the above steps.

Step 12: The comprehensive energy consumption is calculated. The sequences corresponding to the minimum comprehensive energy consumption are selected, and the corresponding sequences are saved.

Step 13: The algorithm is terminated.

The flowchart of the proposed AMOGA-QL algorithm is shown in Fig. 7.

5. Algorithm verification

There are currently no publicly reported examples in the literature on the DLBP-TF. To validate the effectiveness and performance of the proposed algorithm AMOGA-QL, two general disassembly line balancing problems of different scales (P25 and P52) from existing literature are chosen for solving, and comparisons are made with existing algorithms. All experiments in this study are run on the same personal computer, equipped with an AMD Ryzen 7 7840 H processor with Radeon (TM) 780 M Graphics, 3.80 GHz, 32 GB RAM, and Windows 11 64-bit operating system. The algorithm program is written in MATLAB.

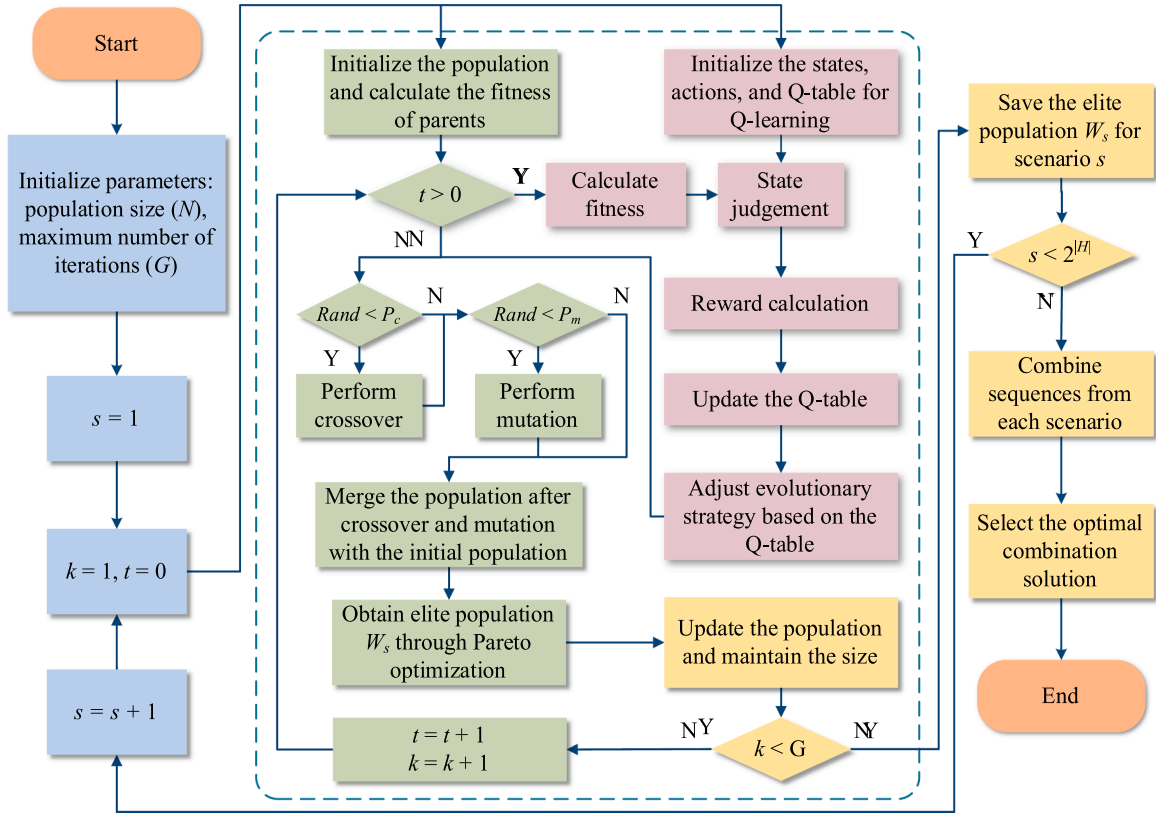


Fig. 7. The flowchart of the proposed AMOGA-QL.

5.1. Mobile phone disassembly case

A classic disassembly case involving 25 disassembly tasks (P25) of the SCH-3500 mobile phone is solved. The precedence relationships of parts, disassembly time parameters, demand parameters, hazard parameters, and cycle time involved in this case are detailed in the literature [49]. The optimization objectives include minimizing the number of workstations f_1 , smoothing index f_2 , demand index f_3 , and hazard index f_4 . The literature [50] proposed a multi-objective genetic algorithm (MOGA) to solve this problem and demonstrated that this algorithm is superior to several multi-objective comparison algorithms. According to the parameters used in that literature, the proposed AMOGA-QL is compared with the MOGA and QL.

Firstly, three algorithms are used to solve the problem with a single objective, and the optimization objective f_2 minimizing. The relevant parameters of GA and AGA-QL are the same. The results of the three algorithms under different maximum iteration times (G) are shown in Table 2. The results show that the proposed AGA-QL performs best when optimizing the objective f_2 independently. The performance of QL is significantly worse than the other two algorithms in standalone problem-solving. These indicate that QL has limited solving ability when used alone. However, as an auxiliary method to improve genetic algorithms, it can effectively enhance computational performance.

As shown in Table 3, the Pareto front of the proposed AMOGA-QL includes all solutions of INSGA-II. The INSGA-II solution set contains ten solutions, while the AMOGA-QL solution set contains 36 solutions, far more than INSGA-II, indicating that AMOGA-QL has greater comprehensiveness. In terms of single objective values, AMOGA-QL can find the known minimum objective values. This demonstrates that the AMOGA-QL is effective and superior to INSGA-II.

In summary, using Q-learning to improve the genetic algorithm yields better results than using either the genetic algorithm or Q-learning alone.

Table 2

Comparison results of GA, QL, and AGA-QL for the P25 case in f_2 .

G	Algorithm	f_2	G	Algorithm	f_2
20	GA	127	50	GA	95
	QL	293		QL	275
	AGA-QL	97		AGA-QL	79
100	GA	12	200	GA	9
	QL	157		QL	89
	AGA-QL	9		AGA-QL	9

5.2. Sewing machine disassembly case

In the literature [51], a case study of sewing machine disassembly with 52 disassembly tasks (P52) is provided and solved using a GA. The three optimization objectives are minimizing idle rate f_1 , maximizing load balancing index f_2 , and minimizing disassembly cost f_3 . The comparison algorithms include ACO (ant colony optimization), AFSA (artificial fish swarm algorithm), and GSA (genetic simulated annealing). The optimization results of the aforementioned algorithms are compared with those of the proposed AMOGA-QL. Each algorithm uses the optimal parameter combination obtained by GA in the literature ($N = 100$, $G = 160$) for independent operations, and the results are shown in Fig. 8. By combining the Pareto fronts of the five algorithms (of which there are 49), 16 new Pareto solutions are selected, among which seven solutions belong to GA (43.75 %) and nine solutions belong to AMOGA-QL (56.25 %), while the solutions of the other three algorithms are not retained. This indicates that the quality of the solutions obtained by AMOGA-QL is better.

To further compare the optimization effect of the algorithms, the minimum, average, and maximum values of the Pareto front obtained by five algorithms on each single objective are compared. Except for ACO achieving values of 0.0579 and 0.1757 on objective f_1 , the solutions for this objective obtained by the other four algorithms are all 0.0579. For

Table 3

Comparison of MOGA and AMOGA-QL results for the P25 case.

Algorithm	No.	f_1	f_2	f_3	f_4	No.	f_1	f_2	f_3	f_4	No.	f_1	f_2	f_3	f_4
MOGA	1	9	9	825	76	5	11	395	870	70	9	12	559	804	71
	2	9	9	823	77	6	11	395	868	71	10	12	559	802	72
	3	9	15	815	75	7	11	433	805	73					
	4	10	111	900	73	8	11	523	802	73					
AMOGA-QL	1	9	9	825	76	13	10	195	813	73	25	11	295	809	73
	2	9	9	823	77	14	10	141	812	74	26	11	433	805	73
	3	9	11	819	76	15	10	141	814	73	27	11	433	807	72
	4	9	11	821	75	16	10	253	809	74	28	11	321	808	73
	5	9	15	815	75	17	10	253	811	73	29	11	379	808	72
	6	9	15	817	74	18	10	217	809	75	30	11	321	810	72
	7	10	113	897	72	19	11	395	868	71	31	11	379	806	73
	8	10	157	874	72	20	11	393	873	71	32	11	343	806	74
	9	10	157	876	71	21	11	395	870	70	33	11	397	805	74
	10	10	155	879	72	22	11	287	810	73	34	12	559	802	72
	11	10	111	900	73	23	11	287	812	72	35	12	559	804	71
	12	10	195	811	74	24	11	295	811	72	36	12	523	802	73

objective f_2 , the optimal value 0.9999 appears in both GA and AMOGA-QL, and the other objective values obtained by AMOGA-QL are better than those obtained by GA in the corresponding solution. For objective f_3 , the maximum, minimum, and average values obtained by AMOGA-QL are all superior to those obtained by the other algorithms, and it achieves the known optimal solution, which is 124.800. These show that the proposed AMOGA-QL achieves effective results for multi-objective optimization and obtains better results than other comparison algorithms.

5.3. Problem expansion for mobile phone disassembly case

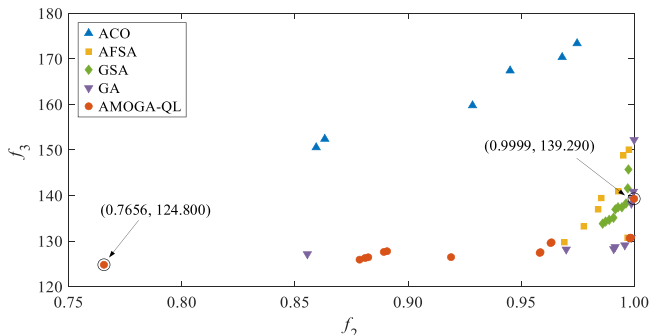
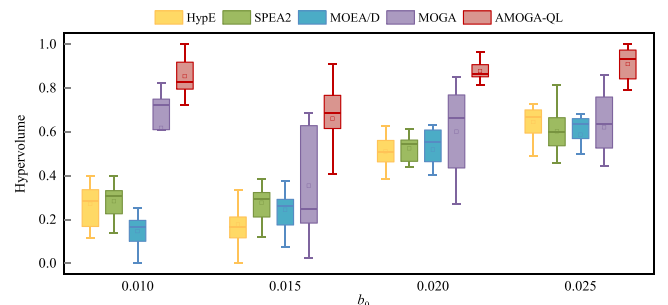
The data from the P25 case is used to discuss the problem in this study, comparison algorithms include HypE (hypervolume based evolutionary algorithm) [52], SPEA2 (strength Pareto evolutionary algorithm 2) [53], and MOEA/D (multi-objective evolutionary algorithm based on decomposition) [54]. At the same time, to verify the role of Q-learning proposed in the algorithm, a multi-objective genetic algorithm (MOGA) without Q-learning is introduced for comparison. In this chapter and subsequent sections, except for the absence of Q-learning, the strategy and parameters of MOGA are the same as those of the proposed AMOGA-QL. The optimization results of the proposed AMOGA-QL are compared with those of the comparison algorithms.

For each algorithm, the population size for both the predictive and rebalancing phases is set to 120, with the termination condition being a runtime of 10 s. Among the first 12 and the last 13 tasks, $h_1 = 9$ and $h_2 = 13$ are randomly selected as hazardous tasks, and the failure probabilities of these hazardous tasks are randomly generated within the [0.1, 0.3] interval, resulting in $f_{h1} = 0.30$ and $f_{h2} = 0.26$. The power settings are $P_1 = 5.0$ kW, $P_2 = 0.5$ kW, and $P_3 = 2.0$ kW. The crossover and mutation probabilities in the comparison algorithm are set to 0.80 and 0.08 respectively. The estimated number of hypervolume points in HypE

is set to 8888.

Nine evaluation indicators are used as objective values, including the number of workstations needed for various scenarios (i.e., the maximum number of workstations required among all of the scenarios), the smoothness index of the predictive phase and three rebalanced sequences under different scenarios, the adjustments in the rebalanced sequences, and the comprehensive energy consumption under each scenario. The HV [55] of the Pareto front of each algorithm is calculated, and boxplots of the normalized results of the four algorithms running 20 times are shown in Fig. 9. Under different parameter conditions, AMOGA-QL outperforms HypE, SPEA2, MOEA/D, and MOGA in terms of minimum, average, and maximum values. MOGA performs well in terms of maximum value but has poor stability. This indicates that the performance of the proposed algorithm is superior to the four comparison algorithms, and the introduction of Q-learning in the proposed algorithm effectively enhances the stability of the algorithm.

Since the problem consists of thousands of balancing optimization processes, the number of possible solutions is enormous, making it impractical to list each solution process. Therefore, one of the optimization results is selected for the analysis. When $s = 1$, each algorithm runs 20 times. The optimization objectives are minimizing the number of workstations in the disassembly line, the smoothness index, and the energy consumption. The results are shown in Table 4. The results show that, under the same initial population size and runtime, the solution set obtained by the proposed AMOGA-QL is superior to those of the other four algorithms according to the definition of Pareto dominance. A random solution is selected as the predicted sequence for rebalancing optimization at h_1 , with optimization objectives including minimizing the number of workstations, the smoothness indices, the adjustments in the rebalanced sequence, and the energy consumption. The Pareto front obtained at the best HV among the 20 runs of each algorithm is shown in Fig. 10. The results indicate that all solutions obtained by AMOGA-QL dominate those of the other algorithms under different parameters,

**Fig. 8.** The results of five different algorithms for the P52 case.**Fig. 9.** Boxplots of the four algorithms with different penalty multipliers.

while the solutions of the other algorithms cannot dominate those of AMOGA-QL.

In terms of single objective values, the solutions of AMOGA-QL perform well. When $b_0 = 0.01$, AMOGA-QL obtains the known minimum smoothness index, adjustments in the rebalanced sequence, and energy consumption, which are 13, 14, and 0.34 kJ, respectively. AMOGA-QL obtains the known minimum smoothness index and energy consumption when $b_0 = 0.015$, which are 13 and 0.41 kJ, respectively. AMOGA-QL obtains the known minimum smoothness index, adjustments in the rebalanced sequence, and energy consumption when $b_0 = 0.02$, which are 13, 11, and 0.455 kJ, respectively. AMOGA-QL obtains the known minimum smoothness index, adjustments in the rebalanced sequence, and energy consumption when $b_0 = 0.025$, which are 16, 14, and 0.55 kJ, respectively. The comparison results indicate that AMOGA-QL's optimization performance for both predicted balance and rebalancing parts is superior to that of the other four algorithms.

6. Engineering case analysis

6.1. Case data

The authors conducted on-site research on the scrap refrigerator dismantling line of a disassembly enterprise. Using the disassembly line as an example, the proposed model and algorithm are applied to analyze the DLBRP-TF. Fig. 11 shows the on-site situation of disassembly lines in enterprises. Fig. 12 shows the disassembly of some parts of the refrigerator. The disassembly tasks are categorized based on the structure and main components of the refrigerator. The complete decomposition diagram of the refrigerator's components is shown in Fig. 13. The specific disassembly task information is shown in Table 5. There are a total of 66 disassembly tasks, and their precedence relationships are illustrated in Fig. 14.

Among the first 33 and last 33 tasks, $h_1 = 18$ and $h_2 = 45$ are randomly selected as hazardous tasks. The failure probabilities for these hazardous tasks are randomly generated within the $[0.1, 0.3]$ interval, resulting in $f_{h1} = 0.13$ and $f_{h2} = 0.21$. The power settings are $P_1 = 6.5$ kW, $P_2 = 0.5$ kW, and $P_3 = 2.0$ kW. The penalty multiplier is set to $b_0 = 0.02$, and the disassembly line cycle time is set to 27 s.

6.2. Algorithms performance comparison

The main parameters of AMOGA-QL include population size (N), number of iterations (G), crossover probability (P_c), mutation probability (P_m), reinforcement learning coefficient (α), and the discount factor-related parameter (δ). The Taguchi experimental design is used to analyze the impact of these four parameters on the performance of AMOGA-QL. The orthogonal array $L_{25}(5^6)$ is shown in Table 6. Nine evaluation indicators are used as objectives, including the number of workstations needed for various scenarios (i.e., the maximum number of workstations required among all of the scenarios), the smoothness index of the predictive phase, and three rebalanced sequences under different scenarios, the adjustments in the three rebalanced sequences, and the comprehensive energy consumption under each scenario. AMOGA-QL is independently run 15 times under each set of parameter conditions, and the average HV of the results is recorded.

To further validate the optimization performance of the proposed AMOGA-QL in the DLBRP-TF, four classic multi-objective evolutionary algorithms (HypE, SPEA2, MOEA/D, and MOGA) are introduced for comparison. According to the parameters and results in Table 6, a main effect plot is drawn as shown in Fig. 15. From Fig. 15, it can be seen that the best parameter combination is $N = 200$, $G = 200$, $P_c = 0.80$, $P_m = 0.10$, $\alpha = 0.9$, and $\delta = 0.1$. The population size and iteration number of the comparison algorithms are also set to 200 and 200 respectively, with the crossover probability and mutation probability set to 0.80 and 0.10. The number of Hypervolume estimation points in HypE is set to 8888.

Each algorithm is independently run 20 times, and the Pareto front of all results is selected as the true Pareto front. The data is normalized, and boxplots of the results of the five algorithms on two evaluation indicators are made as shown in Fig. 16. Comparisons show that the average levels of AMOGA-QL on the HV and IGD [56] indicators are superior to the four comparison algorithms, indicating better convergence and diversity of AMOGA-QL. In terms of stability, the stability of MOEA/D and MOGA is slightly worse, AMOGA-QL has the best stability. The comprehensive comparison shows that AMOGA-QL has an advantage in performance over the four compared algorithms.

The predictive phase and rebalancing phase are further analyzed separately, where f_M is the number of workstations, f_{SI} is the smoothness index, f_V is the adjustments of the rebalanced sequences, and f_E is the comprehensive energy consumption of the disassembly line. When $s = 1$, each algorithm is run 20 times with the optimization objective of minimizing the number of workstations, the smoothness index, and the energy consumption. The results are shown in Table 7. The results indicate that, under the same initial population size and number of runs, AMOGA-QL achieves better solutions in terms of average, maximum, and minimum values. The solutions of the AMOGA-QL are superior to the other four algorithms according to the definition of the Pareto dominance relationship. For the smoothness index and energy consumption, the solutions of AMOGA-QL are significantly better and more uniform.

The rebalancing phase is further analyzed. One solution is randomly selected as the predicted sequence and rebalanced at h_1 . Pareto fronts at the best HV from 20 runs of each algorithm are plotted as scatter plots, as shown in Fig. 17. The comparison shows that the proposed AMOGA-QL solutions are closer to the true Pareto front, and the number of Pareto front solutions in AMOGA-QL is large and can dominate the solutions of the other four algorithms. The results demonstrate that AMOGA-QL outperforms the four comparison algorithms.

To analyze the convergence of the algorithms, the iterative convergence of the hypervolume values for each algorithm is analyzed when $s = 2$. As shown in Fig. 18, the initial solutions of AMOGA-QL are significantly better than those of the four comparison algorithms. During algorithm iteration, AMOGA-QL converges quickly and consistently outperforms the four comparison algorithms in terms of solution quality. AMOGA-QL has the highest HV final convergence value. Therefore, AMOGA-QL's optimization capability is superior to the four comparison algorithms, and the introduction of Q-learning in the proposed algorithm effectively improves the convergence speed and stability of the algorithm.

The introduction of Q-learning makes the algorithm more efficient

Table 4
Optimization results for four algorithms when $s = 1$.

Algorithm	f_1			f_2			f_3		
	max	min	avg	max	min	avg	max	min	avg
HypE	10	10	10.00	81	79	80.5	0.31875	0.31875	0.31875
SPEA2	10	10	10.00	81	79	79.8	0.31875	0.31875	0.31875
MOEA/D	9	10	9.85	81	9	69.3	0.31875	0.30625	0.31625
MOGA	10	9	9.30	79	9	26.7	0.31875	0.30625	0.31000
AMOGA-QL	10	9	9.15	79	10	19.5	0.31875	0.30625	0.30813

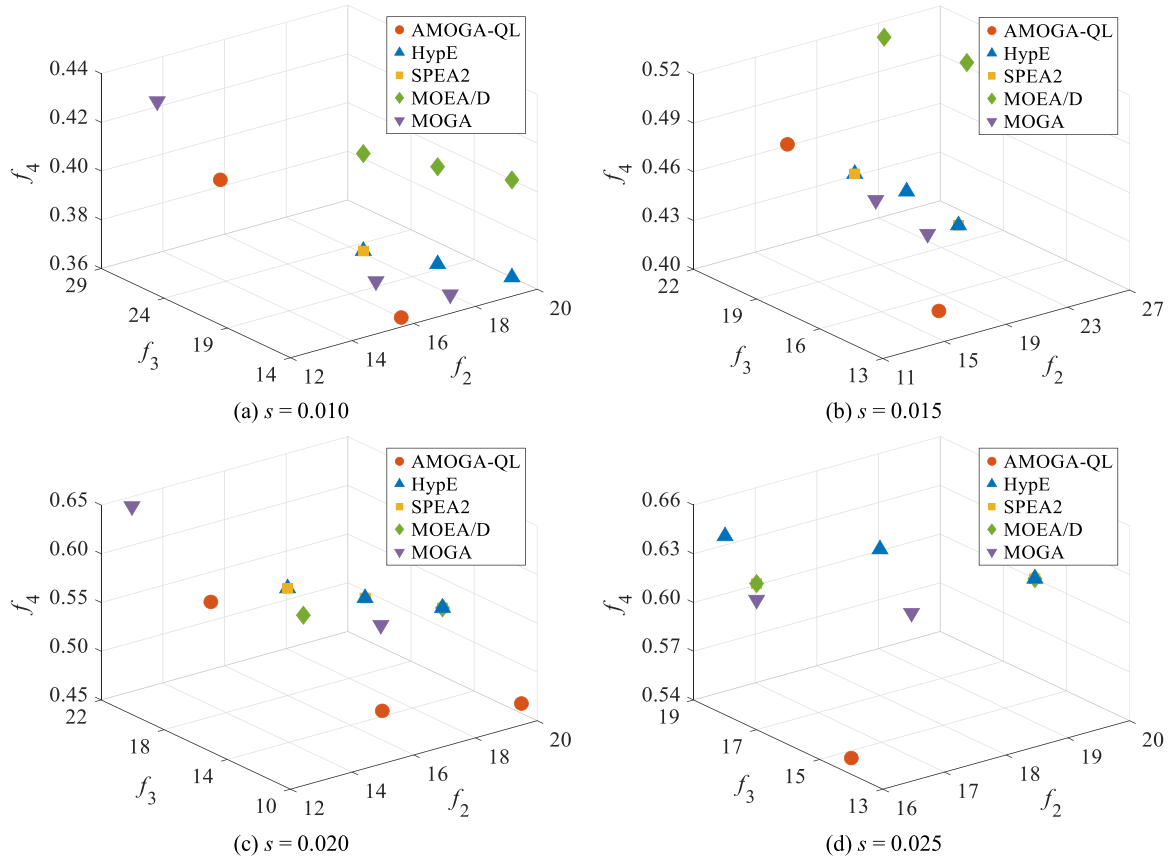


Fig. 10. Optimization results for four algorithms when $s = 2$.



Fig. 11. Disassembly of on-site research scenarios for enterprises.

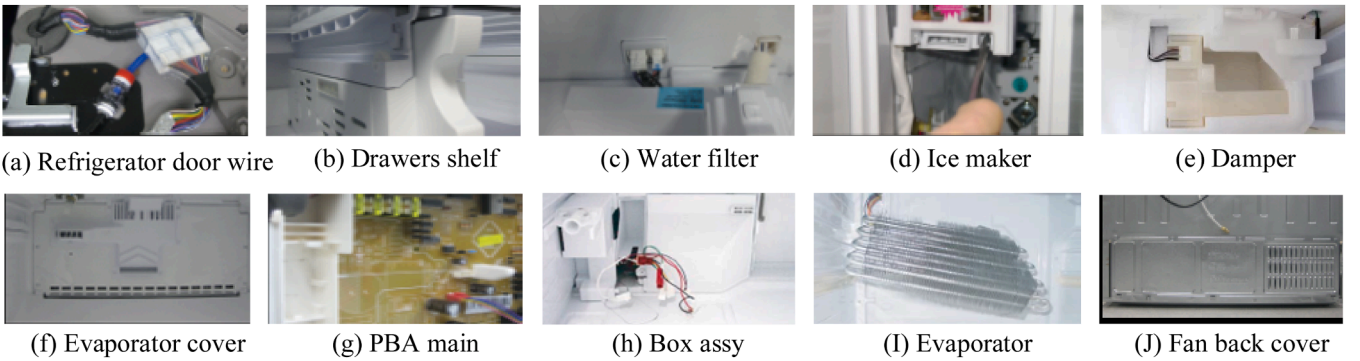


Fig. 12. Some components of the refrigerator.

and high-quality in solving large-scale dynamic problems, proving the effectiveness and superiority of the proposed algorithm in solving such problems.

6.3. Scheme discussion

Table 8 shows the best Pareto front obtained from 20 runs of AMOGA-QL, resulting in 11 combined schemes. The different schemes

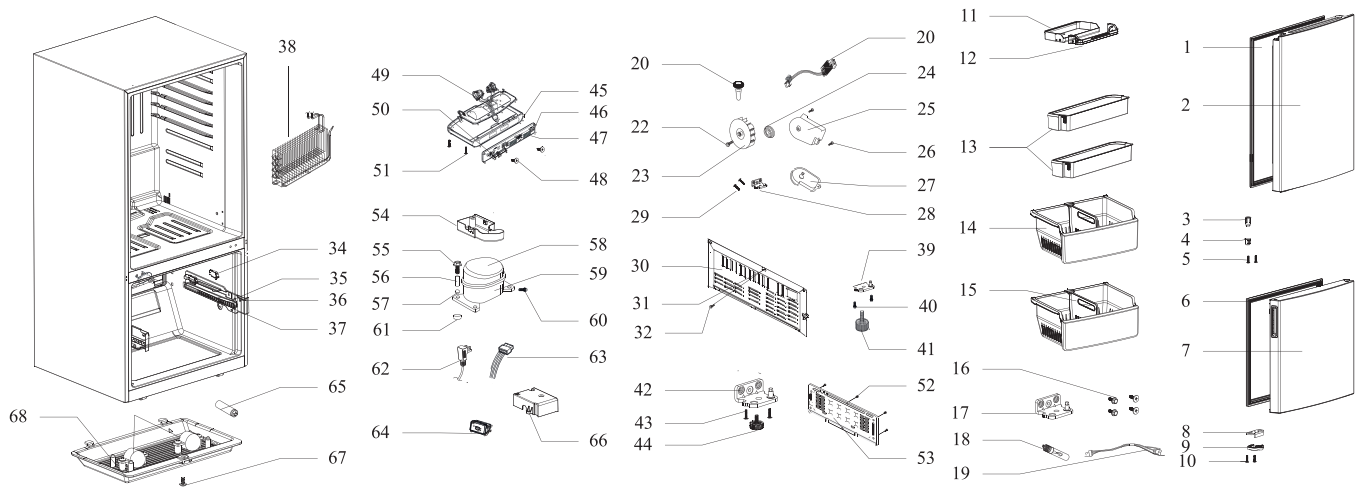


Fig. 13. Exploded view of the refrigerator.

Table 5
The information on refrigerator disassembly tasks.

No.	disassembly tasks (<i>i</i>)	t_i	g_i	No.	disassembly tasks (<i>i</i>)	t_i	g_i	No.	disassembly tasks (<i>i</i>)	t_i	g_i
1	Remove #11#12#13	10	2.1	23	Remove thermostat#20	4	1	45	Remove compressor back cover #53	4	2.4
2	Remove drawer #14	8	2	24	Remove season switch#21	3	0.7	46	Remove water tray#54	5	2.3
3	Remove shelf #15	6	1.2	25	Remove wire#19	3	0.7	47	Remove upper process tube #58	27	5.6
4	Remove screws #16	5	0.3	26	Remove bulb #18	2	0.7	48	Remove hex bolt #55	12	2
5	Remove upper hinge #17	3	0.9	27	Remove screws #26	4	0.3	49	Remove sleeve #56 gasket #57	8	1
6	Remove door #2	5	4.6	28	Remove thermostat box#25	5	1	50	Remove ground screw #60	6	0.4
7	Remove door seal #1	6	1.5	29	Remove screws #36	12	0.6	51	Remove relay cover#59	3	0.4
8	Remove screws #5	4	0.3	30	Remove inner roller#35	12	0.7	52	Remove compressor#58	6	15.8
9	Remove door stopper #4	2	0.4	31	Remove stop block#34	12	0.6	53	Remove washers#61	2	0.3
10	Remove door stopper #3	3	0.9	32	Remove stop bar#37	6	0.7	54	Remove junction box#66	10	2.3
11	Remove screws #29	5	0.3	33	Remove evaporator #38	20	9.7	55	Remove Overload Protector #65	8	1.7
12	Remove middle hinge #28	3	1.1	34	Remove screw cover#32	4	0.4	56	Remove relay#64	8	2.2
13	Remove door #7	5	4.6	35	Remove screws #31	4	0.3	57	Remove relay wire#63	6	1.3
14	Remove door seal #6	6	1.5	36	Remove tube stop#30	3	1.2	58	Remove power cord#62	8	1.8
15	Remove screws #10	4	0.3	37	Remove screws#51	4	0.3	59	Remove screws#67	12	0.3
16	Remove door stopper #9	2	0.4	38	Remove top front cover #45	3	1	60	Remove compressor base plate#68	3	2.6
17	Remove door stopper #8	3	0.9	39	Remove buttons#46	4	0.3	61	Remove leveling feet #44	5	1.2
18	Remove the lampshade #27	2	0.3	40	Remove screws #48	8	0.7	62	Remove screws #43	8	0.8
19	Remove screws #22	2	0.2	41	Remove display board #47	6	4.1	63	Remove hinge #42	3	1.1
20	Remove thermostat knob #23	2	0.4	42	Remove PCB wires#49	4	0.8	64	Remove calibration feet #41	5	1.1
21	Dismantle the gears #24	2	0.2	43	Remove door light switch #50	5	1.1	65	Remove screws #40	4	0.3
22	Remove screws #33	4	0.3	44	Remove screws #52	10	0.7	66	Remove reinforcement key #39	3	1

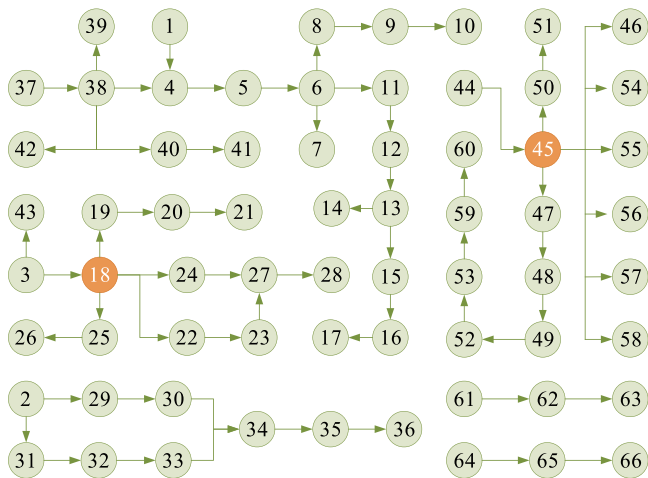


Fig. 14. The precedence relationships of the refrigerator disassembly tasks.

analyzed in Table 8 offer distinct advantages, allowing managers to tailor the schemes based on specific operational goals.

Through optimization, task reallocation between workstations after failure is minimized, with Schemes 3 and 10 achieving zero adjustment in scenarios $s = 3$ and $s = 2$, respectively, reducing adjustment costs. Among all schemes, Scheme 3 has the smallest comprehensive adjustment, while Scheme 4 has the largest. From the perspective of reducing adjustment costs, Scheme 3 is the preferred choice. Energy consumption in the optimized schemes is similar to baseline conditions ($s = 1$). As shown in Fig. 19, Scheme 11 has the highest increase in energy consumption (10.10 %), while Scheme 3 has the lowest (1.61 %). Some schemes, like Schemes 1, 5, and 6, require additional workstations but still maintain lower overall energy consumption by distributing tasks more efficiently. This highlights that the number of workstations is not always linked to higher costs.

Scheme 3 represents an optimal balance between cost and performance. The Gantt charts for scheme 3 are plotted as shown in Fig. 20 (DA: disassembly again). As shown in Fig. 19, in this case, the task adjustments caused by task failures can be completed in the opened workstation, saving the cost of workstation opening and operation. Additionally, combined with Fig. 19, since the load of each workstation is relatively balanced, the scheme with a large adjustment amount is

Table 6

The orthogonal array of AMOGA-QL parameters.

No.	N	G	P_c	P_m	α	δ	HV
1	25	25	0.70	0.05	0.1	0.05	8.840×10^{25}
2	25	50	0.75	0.10	0.3	0.10	8.995×10^{25}
3	25	100	0.80	0.15	0.5	0.15	8.948×10^{25}
4	25	150	0.85	0.20	0.7	0.20	8.962×10^{25}
5	25	200	0.90	0.25	0.9	0.25	9.035×10^{25}
6	50	25	0.75	0.15	0.7	0.25	8.914×10^{25}
7	50	50	0.80	0.20	0.9	0.05	9.005×10^{25}
8	50	100	0.85	0.25	0.1	0.10	9.036×10^{25}
9	50	150	0.90	0.05	0.3	0.15	9.141×10^{25}
10	50	200	0.70	0.10	0.5	0.20	9.117×10^{25}
11	100	25	0.80	0.25	0.3	0.20	8.902×10^{25}
12	100	50	0.85	0.05	0.5	0.25	9.027×10^{25}
13	100	100	0.90	0.10	0.7	0.05	9.039×10^{25}
14	100	150	0.70	0.15	0.9	0.10	9.162×10^{25}
15	100	200	0.75	0.20	0.1	0.15	9.173×10^{25}
16	150	25	0.85	0.10	0.9	0.15	9.085×10^{25}
17	150	50	0.90	0.15	0.1	0.20	9.017×10^{25}
18	150	100	0.70	0.20	0.3	0.25	9.055×10^{25}
19	150	150	0.75	0.25	0.5	0.05	9.070×10^{25}
20	150	200	0.80	0.05	0.7	0.10	9.301×10^{25}
21	200	25	0.90	0.20	0.5	0.10	9.074×10^{25}
22	200	50	0.70	0.25	0.7	0.15	9.068×10^{25}
23	200	100	0.75	0.05	0.9	0.20	9.098×10^{25}
24	200	150	0.80	0.10	0.1	0.25	9.220×10^{25}
25	200	200	0.85	0.15	0.3	0.05	9.177×10^{25}

often accompanied by high energy consumption, that is, the adjustment amount plays an important role in the comprehensive energy consumption.

To assess the practical significance of the proposed scheme, the disassembly line revenue is compared between the rebalancing method and the partial disassembly method. Using the method from [35] and

AMOGA-QL, the maximum revenue from refrigerator disassembly is calculated. The results show that rebalancing generates 97.770 Yuan, while partial disassembly yields 87.360 Yuan. Rebalancing restores 11.98 % more revenue, demonstrating its effectiveness in recovering disassembly line operations.

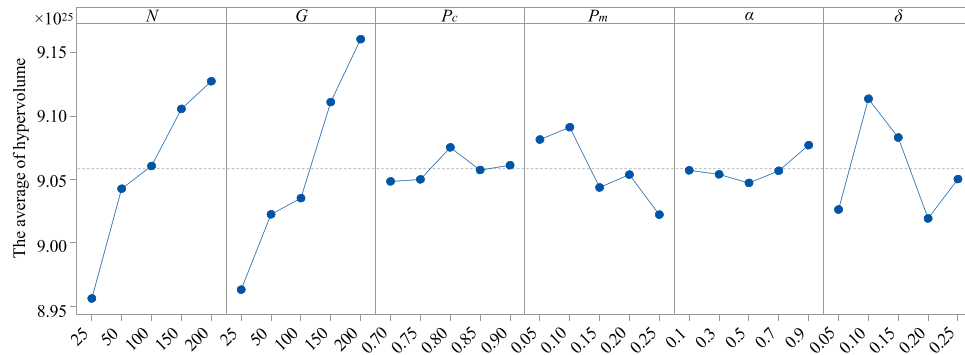
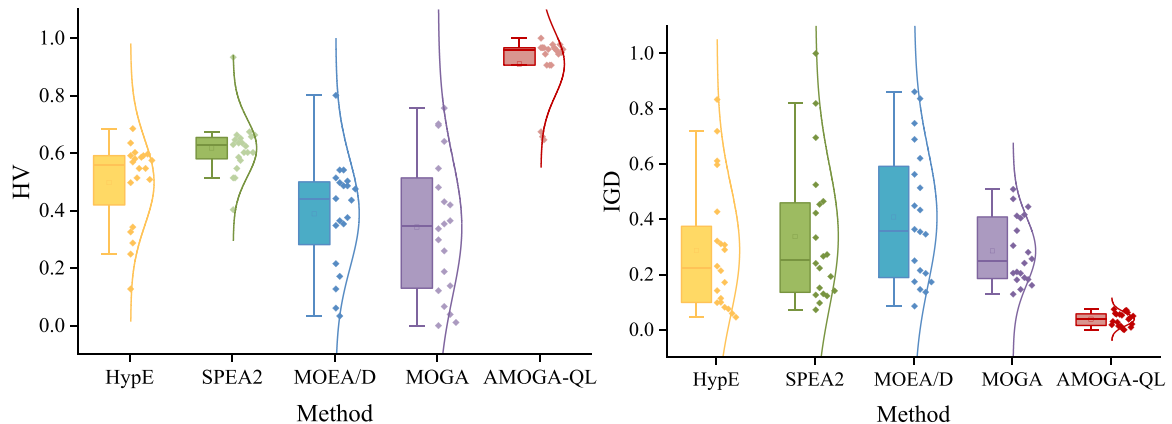
6.4. Managerial inspiration

This study can provide important insights for managers, especially in improving dismantling efficiency, reducing risks, and optimizing resource allocation. Here are several important inspirations:

- (1) Management and response to the risk of task failure: Task failures during disassembly can significantly impact efficiency. The proposed mathematical model enables managers to create backup plans based on hazardous tasks and their failure probabilities, allowing the disassembly line to quickly adapt to the disruptions and minimize the impact.
- (2) Disassembly line balancing and resource optimization: The model of DLBRP-TF and the AMOGA-QL helps managers re-examine the rational allocation of disassembly tasks. Managers can avoid

Table 7Optimization results of four algorithms when $s = 1$.

Algorithm	f_M	f_{SI}			f_E		
		min	max	avg	min	max	avg
HypE	15	31	148	38.55	0.925	0.943	0.926
SPEA2	15	31	148	50.15	0.925	0.943	0.927
MOEA/D	15	33	47	37.20	0.925	0.925	0.925
MOGA	15	29	33	29.40	0.925	0.925	0.925
AMOGA-QL	15	29	31	29.30	0.925	0.925	0.925

**Fig. 15.** The main effects plot for AMOGA-QL.**Fig. 16.** The boxplot of five algorithms on two indicators.

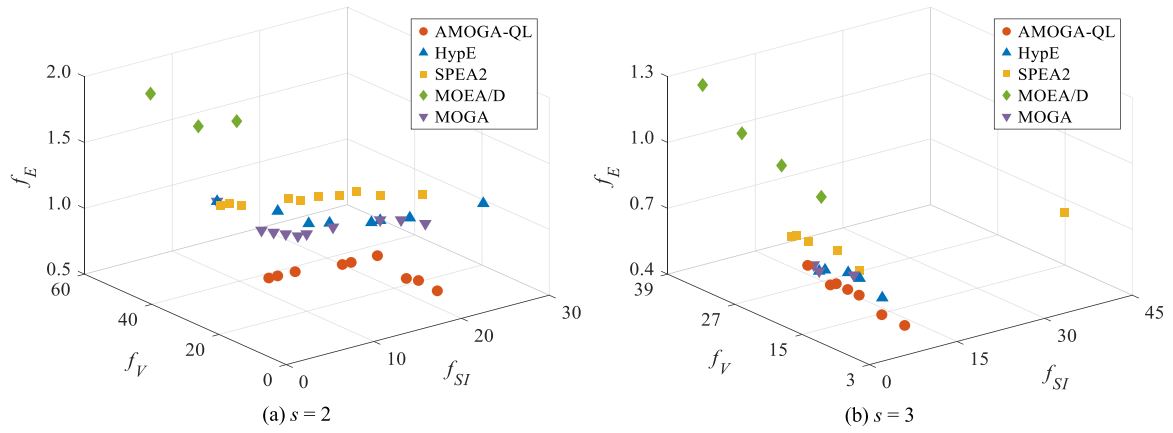


Fig. 17. Optimization results of five algorithms after one rebalancing.

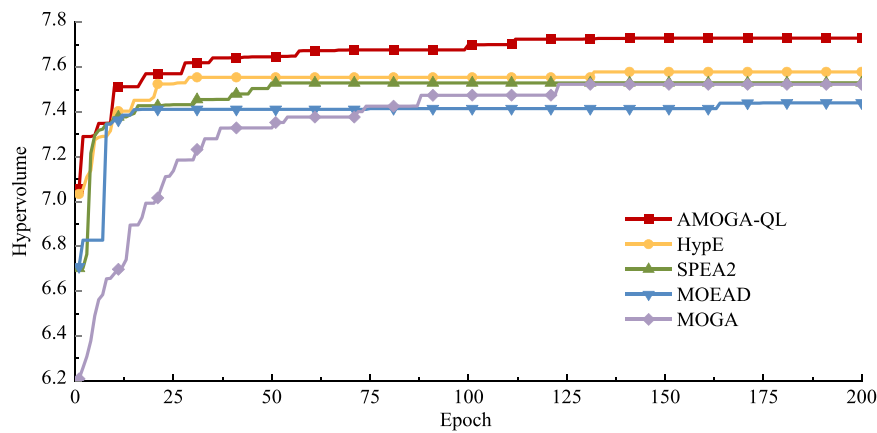


Fig. 18. Iterative curves of four algorithms.

Table 8
Optimization results of AMOGA-QL.

No.	f_M	$s = 1$	$s = 2$		$s = 3$		$s = 4$		f_E
		f_{SI}	f_{SI}	f_V	f_{SI}	f_V	f_{SI}	f_V	
1	16	29	15	7	15	8	16	9	0.9759
2	15	29	15	9	15	11	15	18	0.9952
3	15	31	15	4	15	0	15	10	0.9399
4	15	29	15	21	15	8	15	31	1.0154
5	16	33	15	2	15	9	16	5	0.9671
6	16	29	15	21	15	7	16	26	1.0105
7	15	29	15	31	15	1	15	40	1.0151
8	15	29	15	15	15	10	15	26	1.0076
9	15	29	15	29	15	2	15	31	1.0100
10	15	31	15	0	15	9	15	14	0.9672
11	15	29	15	20	15	9	15	33	1.0184

concentrating too many tasks on a single workstation, ensure the reasonable distribution of resources in each link, and minimize adjustments after task failures. The study shows that disassembly energy consumption is not directly proportional to the number of workstations when considering task failure, managers should consider factors like line distribution, site requirements, and equipment costs to select a plan that best suits their enterprise.

- (3) Profit recovery and cost reduction: The BLBRP-TF helps companies prevent incomplete disassembly due to task failures, recover losses, and reduce waste. It enables managers to rearrange task sequences and adjust workstation loads, minimizing time and cost waste, improving efficiency, and lowering production costs.

- (4) Sustainable development: The disassembly of e-waste is not only a production efficiency issue but also an environmental management issue. Managers can reduce the energy consumption of the disassembly line while minimizing waste and resource wastage caused by task failures and enhancing the environmental friendliness of the disassembly process. This aligns with sustainable development goals, enhances corporate social responsibility, and supports green development trends.

When selecting a specific disassembly scheme, managers should assess actual needs, including energy requirements, cost constraints, corporate goals, etc. Appropriate solutions will help optimize efficiency and profitability and ensure sustainable resource and energy management.

7. Conclusion

This study provides a way of thinking about the disassembly design of e-waste, taking into account the possible task failures that may occur during the disassembly process. This study introduces a DLBRP-TF model. The number of workstations, smoothness index, sequence adjustments, and energy consumption are used to evaluate performance. To ensure efficiency, an AMOGA-QL is proposed, and six evolutionary actions and four population performance states are designed to obtain superior solutions. Q-learning is employed to select the optimal evolutionary operators that adapt the algorithm to the problem's characteristics. The proposed algorithm can generate disassembly scheme combinations for various scenarios, based on known hazardous tasks and their failure probabilities.

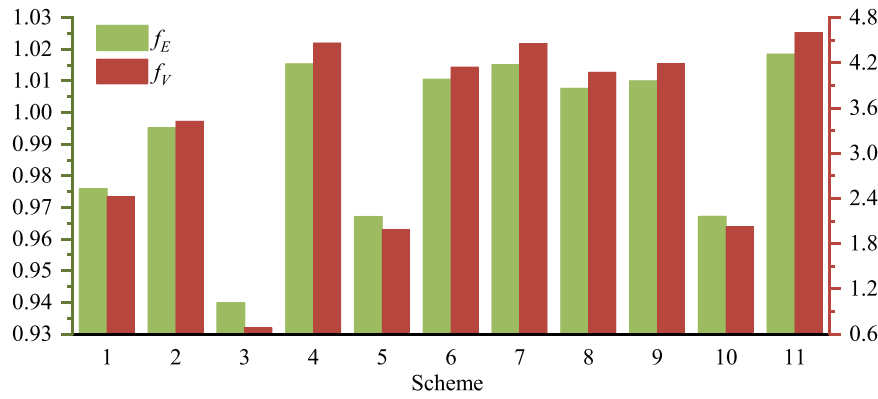


Fig. 19. Comprehensive parameters of each disassembly scheme.

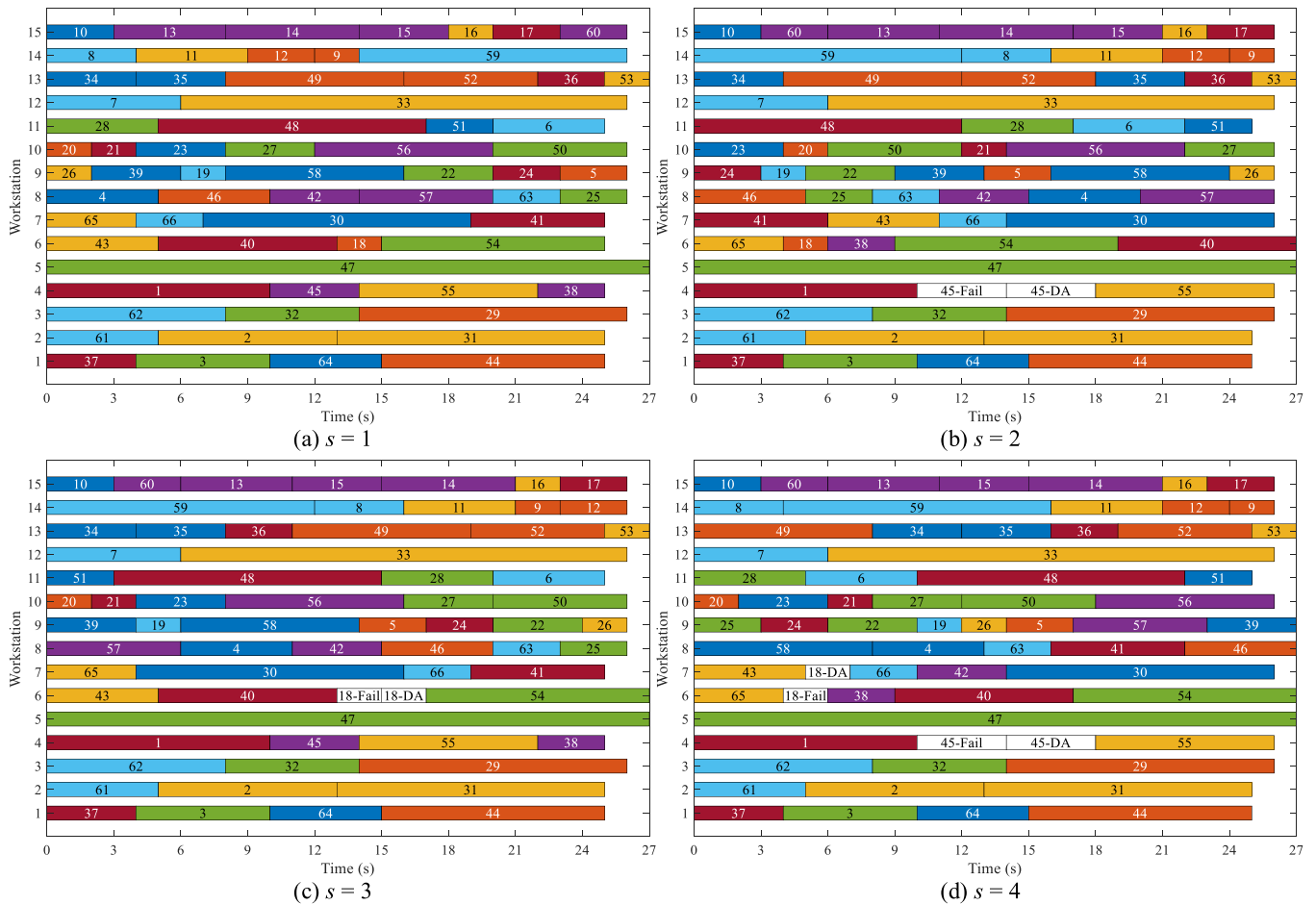


Fig. 20. Gantt charts for scheme 3 under each scenario.

The model and algorithm are applied to a refrigerator disassembly line case with two hazardous tasks, and a total of 11 disassembly schemes are obtained. A scheme combination with minimal energy consumption is developed, and rebalancing is shown to restore 11.98 % of the disassembly line's efficiency. At the same time, the method proposed in this study indicates that it is possible to maintain a stable production process at minimal cost by optimizing and reducing disassembly line adjustments caused by rebalancing. These indicate that the proposed model and algorithm can effectively solve the dynamic disassembly problem of products by designing more efficient, practical, and energy-saving disassembly solutions. The results of this study provide more options for dismantling enterprises, providing theoretical and

technical support for the dismantling design of e-waste. Managers can customize their approach based on specific operational goals, cost constraints, and energy consumption priorities.

Future work could include more hazardous tasks and further testing under varying failure probabilities for a more comprehensive analysis. Intelligent sensing technology can be used to monitor task failures in real-time, and combined with automation equipment for rapid adjustment, gradually promoting the upgrading of disassembly technology.

CRediT authorship contribution statement

Ma Xiaoyi: Writing – original draft, Visualization, Validation,

Software. **Wang Kaipu**: Writing – review & editing, Supervision, Methodology, Formal analysis, Conceptualization. **Luo Yabo**: Writing – review & editing, Resources, Project administration. **Li Yibing**: Resources, Project administration, Formal analysis, Conceptualization. **Gao Liang**: Writing – review & editing. **Li Yingli**: Methodology, Investigation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the National Natural Science Foundation of China (Grant No. 52305552), the Key Research and Development Projects of Hubei Province (Grant No. 2023BAB193), the Hubei Provincial Natural Science Foundation of China (Grant Nos. 2023AFB138 and 2023AFB088), and the Knowledge Innovation Program of Wuhan-Shuguang Project (Grant No. 2023020201020322).

Data availability

Data will be made available on request.

References

- [1] C.P. Baldé, V. Forti, V. Gray, R. Kuehr, P. Stegmann, The global e-waste monitor, United Nations University (UNU), International Telecommunication Union (ITU) & International Solid Waste Association (ISWA), Bonn/Geneva/Vienna, (2024) 1–109.
- [2] Zhu L, Zhang Z, Guan C. Multi-objective partial parallel disassembly line balancing problem using hybrid group neighbourhood search algorithm. *J Manuf Syst* 2020; 56:252–69. <https://doi.org/10.1016/j.jmsy.2020.06.013>.
- [3] Singh RK, Singh AR, Yadav RK. A balanced-quantum inspired evolutionary algorithm for solving disassembly line balancing problem. *Appl Soft Comput* 2023; 132:109840. <https://doi.org/10.1016/J.ASOC.2022.109840>.
- [4] Yin T, Zhang Z, Wu T, Zeng Y, Zhang Y, Liu J. Multimanned partial disassembly line balancing optimization considering end-of-life states of products and skill differences of workers. *J Manuf Syst* 2023;66:107–26. <https://doi.org/10.1016/J.JMSY.2022.12.002>.
- [5] Kenger ZD, Koc C, Ozceylan E. Integrated disassembly line balancing and routing problem. *Int J Prod Res* 2020;58:7250–68. <https://doi.org/10.1080/00207543.2020.1740346>.
- [6] Guo L, Zhang Z, Zhang X. Human-robot collaborative partial destruction disassembly sequence planning method for end-of-life product driven by multi-failures. *Adv Eng Inf* 2023;55:101821. <https://doi.org/10.1016/J.AEI.2022.101821>.
- [7] Xu W, Cui J, Liu B, Liu J, Yao B, Zhou Z. Human-robot collaborative disassembly line balancing considering the safe strategy in remanufacturing. *J Clean Prod* 2021; 324:129158. <https://doi.org/10.1016/J.JCLEPRO.2021.129158>.
- [8] Zhu L, Zhang Z, Wang Y, Cai N. On the end-of-life state oriented multi-objective disassembly line balancing problem. *J Intell Manuf* 2020;31:1403–28. <https://doi.org/10.1007/s10845-019-01519-3>.
- [9] Huang Y, Luo A, Chen T, Zhang M, Ren B, Song Y. When architecture meets RL+EA: a hybrid intelligent optimization approach for selecting combat system-of-systems architecture. *Adv Eng Inf* 2023;58:102209. <https://doi.org/10.1016/j.aei.2023.102209>.
- [10] Chu M, Chen W. Human-robot collaboration disassembly planning for end-of-life power batteries. *J Manuf Syst* 2023;69:271–91. <https://doi.org/10.1016/j.jmsy.2023.06.014>.
- [11] Jiang T, Liu L, Zhu H. A Q-learning-based biology migration algorithm for energy-saving flexible job shop scheduling with speed adjustable machines and transporters. *Swarm Evol Comput* 2024;90:101655. <https://doi.org/10.1016/j.eswa.2023.120837>.
- [12] Wang J, Tang H, Lei D. A Q-learning artificial bee colony for distributed assembly flow shop scheduling with factory eligibility, transportation capacity and setup time. *Eng Appl Artif Intell* 2023;123:106230. <https://linkinghub.elsevier.com/retrieve/pii/S0952197623004141>.
- [13] Altekin FT, Akkan C. Task-failure-driven rebalancing of disassembly lines. *Int J Prod Res* 2012;50:4955–76. <https://doi.org/10.1080/00207543.2011.616915>.
- [14] Yang C, Gao J, Sun L. A multi-objective genetic algorithm for mixed-model assembly line rebalancing. *Comput Ind Eng* 2013;65:109–16. <https://doi.org/10.1016/j.cie.2011.11.033>.
- [15] Sancı E, Azizoglu M. Rebalancing the assembly lines: exact solution approaches. *Int J Prod Res* 2017;55:5991–6010. <https://doi.org/10.1080/00207543.2017.1319583>.
- [16] Mete S, Cil ZA, Ozceylan E, Agpak K, Battaia O. An optimisation support for the design of hybrid production lines including assembly and disassembly tasks. *Int J Prod Res* 2018;56:7375–89. <https://doi.org/10.1080/00207543.2018.1428774>.
- [17] Ren Y, Zhang C, Zhao F, Tian G, Lin W, Meng L, Li H. Disassembly line balancing problem using interdependent weights-based multi-criteria decision making and 2-optimal algorithm. *J Clean Prod* 2018;174:1475–86. <https://doi.org/10.1016/j.jclepro.2017.10.308>.
- [18] Mete S, Cil ZA, Celik E, Ozceylan E. Supply-driven rebalancing of disassembly lines: a novel mathematical model approach. *J Clean Prod* 2019;213:1157–64. <https://doi.org/10.1016/j.jclepro.2018.12.265>.
- [19] Şahin M, Kellegöz T. A new mixed-integer linear programming formulation and particle swarm optimization based hybrid heuristic for the problem of resource investment and balancing of the assembly line with multi-manned workstations. *Comput Ind Eng* 2019;133:107–20. <https://doi.org/10.1016/j.cie.2019.04.056>.
- [20] Wang K, Li X, Gao L, Garg A. Partial disassembly line balancing for energy consumption and profit under uncertainty. *Rob Comput Integr Manuf* 2019;59: 235–51. <https://doi.org/10.1016/j.rcim.2019.04.014>.
- [21] Yang Y, Yuan G, Zhuang Q, Tian G. Multi-objective low-carbon disassembly line balancing for agricultural machinery using MDOFA and fuzzy AHP. *J Clean Prod* 2019;233:1465–74. <https://doi.org/10.1016/j.jclepro.2019.06.035>.
- [22] Wang K, Li X, Gao L, Li P, Gupta SM. A genetic simulated annealing algorithm for parallel partial disassembly line balancing problem. *Appl Soft Comput* 2021;107: 107404. <https://doi.org/10.1016/j.asoc.2021.107404>.
- [23] Guo X, Zhang Z, Qi L, Liu S, Tang Y, Zhao Z. Stochastic hybrid discrete grey wolf optimizer for multi-objective disassembly sequencing and line balancing planning in disassembling multiple products. *IEEE Trans Autom Sci Eng* 2022;19:1744–56. <https://doi.org/10.1109/TASE.2021.3133601>.
- [24] Lounes BM, Pascale M, Alexandre V, Néjib M. End-of-Life product quality management for efficient design of disassembly lines under uncertainty. *Int J Prod Res* 2023;61:1146–67. <https://doi.org/10.1080/00207543.2022.2028199>.
- [25] Zhang X, Tian G, Fathollahi-Fard AM, Pham DT, Li Z, Pu Y, Zhang T. A chance-constraint programming approach for a disassembly line balancing problem under uncertainty. *J Manuf Syst* 2024;74:346–66. <https://doi.org/10.1016/j.jmsy.2024.03.014>.
- [26] Singh RK, Singh AR, Yadav RK. A station crashing-based recursive approach for disassembly line balancing problem in the presence of task failure. *Int J Prod Res* 2023;61:5659–75. <https://doi.org/10.1080/00207543.2022.2110017>.
- [27] Goksoy Kalaycilar E, Azizoglu M, Batun S. Disassembly line balancing with hazardous task failures – model based solution approaches. *Comput Ind Eng* 2024; 190:110089. <https://doi.org/10.1016/J.CIE.2024.110089>.
- [28] Guo J, Li Y, Du B, Sun X, Wang K. A multi-population cooperative coevolution artificial bee colony algorithm for partial multi-robotic disassembly line balancing problem considering preventive maintenance scenarios. *Adv Eng Inf* 2024;62: 102750. <https://doi.org/10.1016/J.AEI.2024.102750>.
- [29] Hu P, Chu F, Dolgui A, Chu C, Liu M. Integrated multi-product reverse supply chain design and disassembly line balancing under uncertainty. *Omega* 2024;126: 103062. <https://doi.org/10.1016/J.OMEGA.2024.103062>.
- [30] Ren Y, Gao K, Fu Y, Li D, Suganthan PN. Ensemble artificial bee colony algorithm with Q-learning for scheduling Bi-objective disassembly line. *Appl Soft Comput* 2024;155:111415. <https://doi.org/10.1016/j.asoc.2024.111415>.
- [31] Wang J, Xi G, Guo X, Liu S, Qin S, Han H. Reinforcement learning for hybrid disassembly line balancing problems. *Neurocomputing* 2024;569:127145. <https://doi.org/10.1016/j.neucom.2023.127145>.
- [32] Yeni FB, Cevikcan E, Yazici B, Yilmaz OF. Aggregated planning to solve multi-product multi-period disassembly line balancing problem by considering multi-manned stations: a generic optimization model and solution algorithms. *Comput Ind Eng* 2024;196:110464. <https://doi.org/10.1016/j.cie.2024.110464>.
- [33] Zhang C, Chen M. Designing and verifying a disassembly line approach to cope with the upsurge of end-of-life vehicles in China. *Waste Manag (Oxf)* 2018;76: 697–707. <https://doi.org/10.1016/j.wasman.2018.02.031>.
- [34] Guo H, Zhang L, Ren Y, Li Y, Zhou Z, Wu J. Optimizing a stochastic disassembly line balancing problem with task failure via a hybrid variable neighborhood descent-artificial bee colony algorithm. *Int J Prod Res* 2023;61:2307–21. <https://doi.org/10.1080/00207543.2022.2069524>.
- [35] Goksoy Kalaycilar E, Batun S, Azizoglu M. A stochastic programming approach for the disassembly line balancing with hazardous task failures. *Int J Prod Res* 2022; 60:3237–62. <https://doi.org/10.1080/00207543.2021.1916119>.
- [36] Lailli Y, Li X, Wang Y, Ren L, Wang X. Robotic disassembly sequence planning with backup actions. *IEEE Trans Autom Sci Eng* 2022;19:2095–107. <https://doi.org/10.1109/ISSE.2001.924499>.
- [37] Yin T, Zengqiang Z, Tengfei W. Multimanned partial disassembly line balancing optimization considering end-of-life states of products and skill differences of workers. *J Manuf Syst* 2023;66:107–26. <https://doi.org/10.1016/J.JMSY.2022.12.002>.
- [38] Jiang Y, Xu H, Liu Q, Pham DT. Evolutionary optimization using epsilon method for resource-constrained multi-robotic disassembly line balancing. *J Manuf Syst* 2020; 56:392–413. <https://doi.org/10.1016/j.jmsy.2020.06.006>.
- [39] Zhang L, Zhao X, Ke Q, Dong W, Zhong Y. Disassembly line balancing optimization method for high efficiency and low carbon emission. *Int J Precis Eng Manuf -Green Technol* 2021;8:233–47. <https://doi.org/10.1007/s40684-019-00140-2>.
- [40] Chu M, Chen W. Multi-manned disassembly line balancing problems for retired power batteries based on hyper-heuristic reinforcement. *Comput Ind Eng* 2024; 194:110400. <https://doi.org/10.1016/j.cie.2024.110400>.

- [41] Tuncel E, Zeid A, Kamarthi S. Solving large scale disassembly line balancing problem with uncertainty using reinforcement learning. *J Intell Manuf* 2014;25: 647–59. <https://doi.org/10.1007/s10845-012-0711-0>.
- [42] Tang H, Xiao Y, Zhang W, Lei D, Wang J, Xu T. A DQL-NSGA-III algorithm for solving the flexible job shop dynamic scheduling problem. *Expert Syst Appl* 2024; 237:121723. <https://doi.org/10.1016/J.ESWA.2023.121723>.
- [43] Qu W, Li J, Zhang R, Liu S, Bao J. Adaptive planning of human–robot collaborative disassembly for end-of-life lithium-ion batteries based on digital twin. *J Intell Manuf* 2024;35:2021–43. <https://doi.org/10.1007/s10845-023-02081-9>.
- [44] Rahman HF, Janardhanan MN, Ponnambalam SG. Energy aware semi-automatic assembly line balancing problem considering ergonomic risk and uncertain processing time. *Expert Syst Appl* 2023;231:120737. <https://doi.org/10.1016/j.eswa.2023.120737>.
- [45] Song Y, Wei L, Yang Q, Wu J, Xing L, Chen Y. RL-GA: a reinforcement learning-based genetic algorithm for electromagnetic detection satellite scheduling problem. *Swarm Evol Comput* 2023;77:101236. <https://doi.org/10.1016/J.SWEVO.2023.101236>.
- [46] Kalyanmoy D. A fast and elitist multi-objective genetic algorithm: NSGA-II. *IEEE Trans Evol Comput* 2002;6:182–97. <https://doi.org/10.1109/4235.996017>.
- [47] D.A. Van Veldhuizen, Multiobjective evolutionary algorithms: classifications, analyses, and new innovations, Air Force Institute of Technology, 1999.
- [48] Wang Y-N, Wu L-H, Yuan X-F. Multi-objective self-adaptive differential evolution with elitist archive and crowding entropy-based diversity measure. *Soft Comput* 2010;14:193–209. <https://doi.org/10.1007/s00500-008-0394-9>.
- [49] Kalayci CB, Gupta SM. Artificial bee colony algorithm for solving sequence-dependent disassembly line balancing problem. *Expert Syst Appl* 2013;40: 7231–41. <https://doi.org/10.1016/j.eswa.2013.06.067>.
- [50] Li Z, Kucukkoc I, Zhang Z. Iterated local search method and mathematical model for sequence-dependent U-shaped disassembly line balancing problem. *Comput Ind Eng* 2019;137:106056. <https://doi.org/10.1016/j.cie.2019.106056>.
- [51] Zhang Z, Li L, Cai N, Jia L. Local neighborhood genetic algorithm for stochastic disassembly line balancing problem. *Comput Integr Manuf Syst* 2019;25:607–18. <https://doi.org/10.13196/j.cims.2019.03.008>.
- [52] Bader J, Zitzler E. HypE: an algorithm for fast hypervolume-based many-objective optimization. *Evol Comput* 2011;19:45–76. https://doi.org/10.1162/EVCO_a_00009.
- [53] E. Zitzler, M. Laumanns, L. Thiele, SPEA2: Improving the strength Pareto evolutionary algorithm, in: *Proceedings of the Conference on Evolutionary Methods for Design, Optimization and Control with Applications to Industrial Problems*, Athens, Greece, 2001, pp. 95–100.
- [54] Zhang Q, Li H. MOEA/D: a multiobjective evolutionary algorithm based on decomposition. *IEEE Trans Evol Comput* 2007;11:712–31. <https://doi.org/10.1109/TEVC.2007.892759>.
- [55] Zitzler E, Thiele L. Multi-Objective evolutionary algorithms: a comparative case study and the strength Pareto approach. *IEEE Trans Evol Comput* 1999;3: 257–71. <https://doi.org/10.1109/4235.797969>.
- [56] Coello CAC, Cortés NC. Solving multiobjective optimization problems using an artificial immune system. *Genet Program Evol Mach* 2005;6:163–90. <https://doi.org/10.1007/s10710-005-6164-x>.